4 EtherCAT

Dieses Kapitel beschäftigt sich mit **EtherCAT**, der Echtzeit-Ethernet Lösung der Firma Beckhoff. Viele Hersteller und Anwender, die ein gemeinsames Interesse an EtherCAT verbindet, haben sich zur ***ETG*** (***EtherCAT Techology Group***) zusammengeschlossen. Derzeit (Stand: 20.05.2008) hat die ETG 732 Mitglieder.

Nach einer allgemeinen Darstellung der Funktionsweise von EtherCAT wird die Einbindung eines EtherCAT-Slaves in ein EtherCAT-Netzwerk behandelt. An einem einfachen Beispiel wird deutlich, wie ein SPS-Programm auf die Ein- und Ausgangsdaten eines EtherCAT-Slaves zugreift. Dieses Zusammenspiel ist bei anderen Kommunikationssystemen sehr ähnlich.

Danach wird genauer betrachtet, welche Protokollelemente zwischen Master und Slave ausgetauscht werden. Zur Aufzeichung des Datenverkehrs wird wieder Wireshark herangezogen.

Die Testmöglichkeiten sind bei EtherCAT ideal. Eine 30-Tage-Version der PC-Softwarelösung **TwinCAT** kann von der Website der Firma Beckhoff (http://www.beckhoff.de/) heruntergeladen werden. **TwinCAT** enthält insbesondere die Referenz-Implementierung eines EtherCAT-Masters und ein Entwicklungssystem für SPS[1]-Programme.

Die praktischen Beispiele dieses Kapitels beschäftigen sich mit dem Zusammenspiel zwischen EtherCAT-Master und EtherCAT-Slave. Der EtherCAT-Master von TwinCAT verwendet eine normale Ethernet-Schnittstelle des PC. Als typischer EtherCAT-Slave dient ein netX-Starterboard oder ein netSTICK. Leser, die im Besitz der entsprechenden Hardware sind, können die Beispiele selbst nachvollziehen und als Ausgangspunkt für eigene Experimente nehmen.

4.1 Überblick

Ein EtherCAT-System besteht aus mindestens einem **EtherCAT-Master** und einem oder mehreren **EtherCAT-Slaves**. Aus Ethernet-Sicht ist ein EtherCAT-System **ein einziger Ethernet-Knoten**. Innerhalb dieses Knotens befindet sich eine Vielzahl von EtherCAT-Slaves.

EtherCAT lässt eine Vielzahl von Topologien zu: Stern, Linie oder Baum sind möglich. Es können preiswerte Ethernet-Patchkabel verwendet werden.

[1] SPS: Speicherprogrammierbare Steuerung

EtherCAT verwendet hierfür einen Master/Slave-Ansatz, um die Hauptaufgabe des Kommunikationssystems, den schnellen Transport der Prozessdaten zu bewerkstelligen.

Folgende Ziele stehen bei EtherCAT im Vordergrund [JANS03a]:

- **Breite Einsetzbarkeit**: Jede Steuerung mit handelsüblichem Ethernet-Controller soll als EtherCAT-Master eingesetzt werden können.
- **Konformität zum Ethernet-Standard**: EtherCAT-Geräte sollen mit anderen Ethernet-Geräten koexistieren. Standard-Komponenten wie Switches sollen einsetzbar sein.
- Das **Spektrum der EtherCAT-Teilnehmer** soll vom komplexen Knoten bis zum Teilnehmer mit 2 Bit Ein- und Ausgangsdaten reichen.
- **Höchstmögliche Effizienz**, d. h. möglichst wenig Overhead beim Transport von Nutzdaten.
- **Kurze Zykluszeiten**: Mit Zykluszeiten unter 100 µs sollen Anwendungen wie das Schließen von Regelkreisen in der Antriebstechnik realisiert werden.
- **Höchste Deterministik**

Die Durchlaufverzögerung beträgt pro Teilnehmer 60 ns. Als Update-Zeiten werden folgende Zeiten genannt [BÜTT04]:

Tabelle 4.1: *Performance-Angaben*

Anwendung	Update-Zeit
1000 digitale E/As	30 µs
200 16-Bit Analog-E/As	50 µs
100 Servo-Achsen	100 µs

Um die schnellen Reaktionszeiten und minimalen Verzögerungszeiten zu ermöglichen, sind bei den Slaves optimierte Bausteine erforderlich. EtherCAT-ASICs sind auf das Senden und Empfangen von Prozessdaten optimiert. Bei besonders einfachen Slaves kann der ASIC sogar ohne Mikrocontroller eingesetzt werden. Der EtherCAT-ASIC führt zu Anschaltkosten, die unter denen eines Ethernet-Controllers liegen.

4.2 Funktionsprinzip

Üblicherweise findet die EtherCAT-Kommunikation innerhalb eines Ethernet-Segments statt. In diesem Fall benutzt EtherCAT den Wert `0x88A4` als Ethernet-Typ, und der EtherCAT-Frame wird direkt als Datenfeld des Ethernet-Rahmens übertragen. Ein EtherCAT-System stellt logisch gesehen einen offenen Ring dar. Der Master schickt am einen Ende Telegramme in den Ring und erhält am anderen Ende bearbeitete Telegramme zurück. Telegramme werden vom ersten Slave zu den folgenden weitergeleitet und vom letzten Slave an den Master gesendet. Die EtherCAT-Slaves verarbeiten den EtherCAT-Frame „on-the-fly". Ein Slave erkennt ein an ihn gerichtetes Kommando und führt es aus. Die Ausgangsdaten für den Slave werden entnommen, und Eingangsdaten des Slaves werden im Frame abgelegt. Während dessen wird das Telegramm, nur um wenige Nanosekunden verzögert, bereits weitergeschickt. Dies setzt den Vollduplex-Modus mit

getrennten Leitungen für beide Übertragungsrichtungen voraus. In der Regel wird bereits die Antwort zum Master zurückgeschickt, während der Master noch seine Anfrage sendet.

Weniger zeitkritische Anwendungen können auch eine EtherCAT-Kommunikation über Subnetze hinweg nutzen. In diesem Fall werden die EtherCAT-Daten im Datenteil eines UDP-Datagramms transportiert, wobei hier `0x88A4` (34980) als **Portnummer** verwendet wird.

4.3 Topologie, Medien

Da ein Ethernet-Kabel getrennte Sende- und Empfangsleitungen besitzt und auch alle EtherCAT-Slaves in der Rückrichtung übertragen können, handelt es sich bei einer Linienstruktur wie in Bild 4.1 eigentlich um einen offenen Ring.

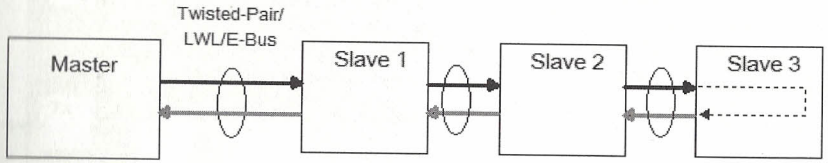


Bild 4.1: *Signalfluss bei einer Linienstruktur. Oben: Hin-Leitung, unten: Rück-Leitung*

Aus dieser Linientopologie kann durch ***Abzweigungen*** eine flexible Baumstruktur aufgebaut werden:

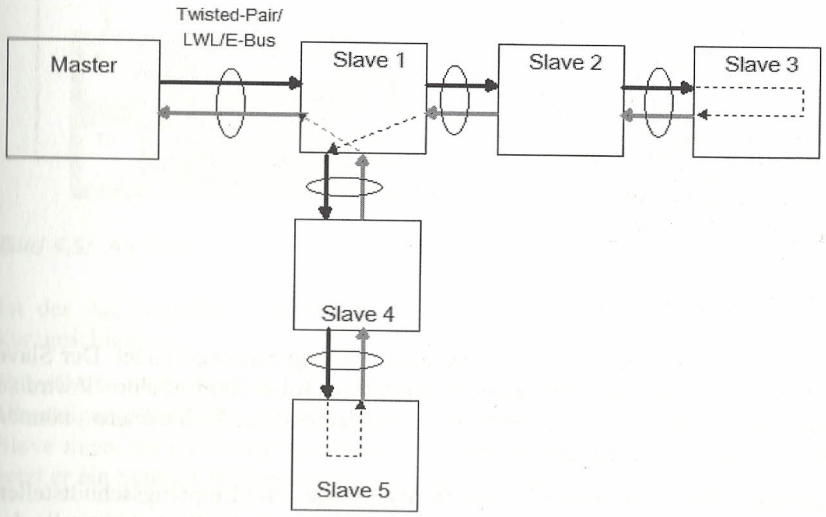


Bild 4.2: *Signalfluss bei einer Baumstruktur, Slave 1 mit Abzweig*

EtherCAT unterstützt verschiedene Medien, u. a. normale **CAT-5-Kupferkabel** und **Lichtwellenleiter** (***LWL***). Häufig hat man sehr kurze Strecken zwischen zwei

Teilnehmern. Hierfür wurde der so genannte ***E-Bus*** entwickelt, der neben der „Klemmen-Kommunikation" auch für kostengünstige Übertragung bis zu 10 m geeignet ist. Der E-Bus nutzt eine LVDS[2]-Übertragung. Es wird immer ein Standard Ethernet-Frame übertragen. Daher kann die Bitübertragungsschicht beliebig oft gewechselt werden.

4.4 Signalfluss

Ein EtherCAT-Slave hat **je zwei** Sendeschnittstellen (Tx) und zwei Empfangsschnittstellen (Rx). Ein vom Master gesendetes Telegramm wird auf der Rx-Schnittstelle der Hin-Leitung empfangen, bearbeitet und auf der Tx-Schnittstelle der Hin-Leitung weitergeleitet:

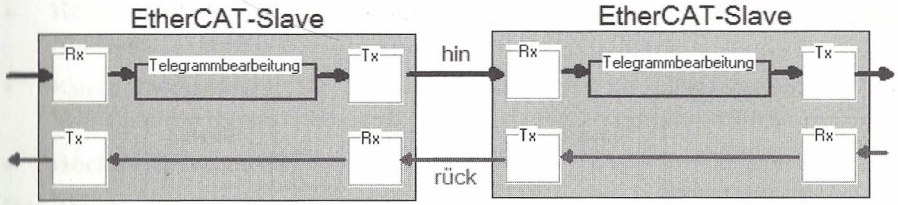


Bild 4.3: *Telegrammbearbeitung*

Ein auf der Rx-Schnittstelle der Rück-Leitung empfangenes Telegramm wird verstärkt und regeneriert. Der Slave erkennt, ob auf der Leitung ein Trägersignal anliegt. Ist auf seiner Rück-Leitung kein Trägersignal mehr, so schaltet er die Tx-Schnittstelle der Hin-Leitung auf die Rx-Schnittstelle der Rück-Leitung:

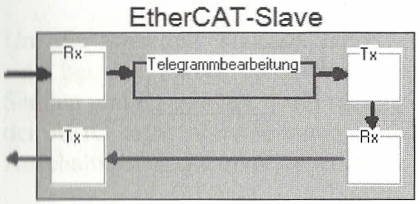


Bild 4.4: *Kein Trägersignal auf der Rück-Leitung*

Damit wird das bearbeitete Telegramm über die Rück-Leitung zurückgesendet. Der Slave erkennt also automatisch, dass nach ihm kein weiterer Slave folgt. Bei Kabelbruch wird so ein funktionsfähiger Ring erreicht. Mit den verbleibenden Teilnehmern können unverändert Prozessdaten ausgetauscht werden.

Bei einem **Abzweig** (Slave 1 in Bild 4.2) sind **je drei** Sende- und Empfangsschnittstellen vorhanden. Das Telegramm geht zunächst **ohne Bearbeitung** von der Rx-Schnittstelle der Hin-Leitung auf die Tx-Schnittstelle der Hin-Leitung. Ein auf der Rx-Schnittstelle der Rück-Leitung empfangenes Telegramm wird bearbeitet und auf die Tx-Schnittstelle des

[2] LVDS: **L**ow **V**oltage **D**ifferential **S**ignaling

Abzweigs gegeben. Ein auf der Rx-Schnittstelle des Abzweigs empfangenes Telegramm wird direkt auf die Tx-Schnittstelle der Rück-Leitung weitergeleitet.

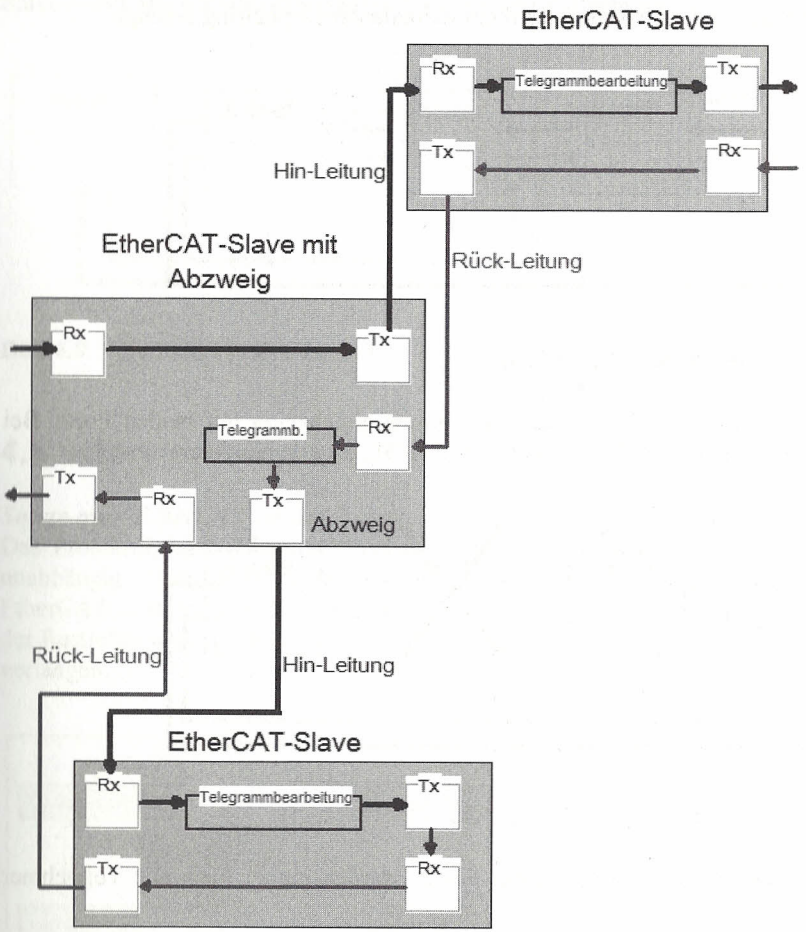


Bild 4.5: *Abzweig*

Ist der Abzweig-Port nicht belegt, so werden Rx- und Tx-Schnittstelle des Abzweigs kurzgeschlossen.

EtherCAT verwendet die normale CRC des Ethernet-Rahmens, um die korrekte Übertragung des Telegramms zu prüfen. Bei jeder Änderung des Telegramms durch einen Slave muss die CRC neu berechnet werden. Erkennt ein Slave eine fehlerhafte CRC, so setzt er ein Status-Bit. Dadurch lässt sich die Fehlerstelle lokalisieren.

4.5 Redundanz

Durch einen zweiten Ethernet-Port beim Master und eine zusätzliche Verbindung zwischen einem Slave und dem Master wird ein redundanter Doppelring erzeugt:

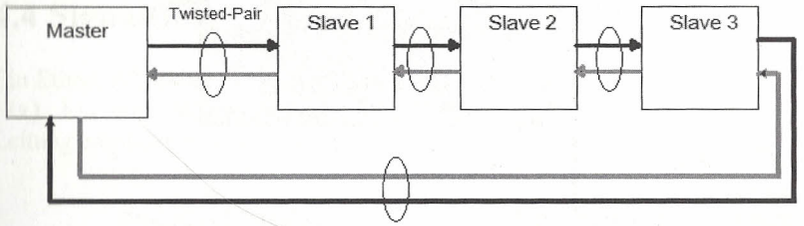


Bild 4.6: *Redundantes System (Doppelring)*

Der Master sendet in diesem Fall seine Telegramme gleichzeitig auf beiden Ports. Bei Ausfall eines Slaves kann weiterhin mit allen übrigen Slaves kommuniziert werden:

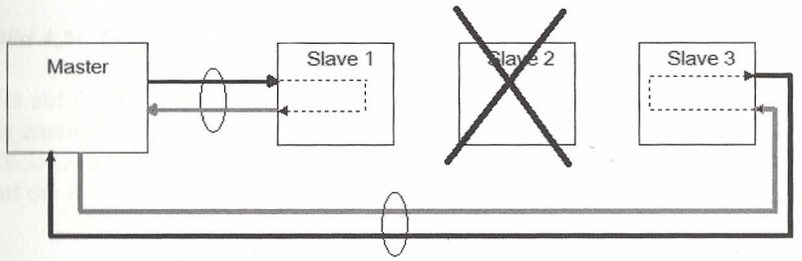


Bild 4.7: *Redundanz: Ausfall eines Teilnehmers*

Auch bei einem Kabelbruch zwischen zwei Slaves werden immer noch alle Teilnehmer erreicht:

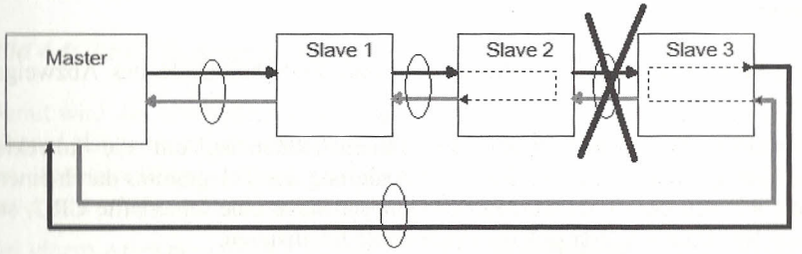


Bild 4.8: *Redundanz: Kabelbruch*

Die zusätzliche Verbindung muss nicht notwendig zwischen dem letzten Slave und dem Master eingerichtet werden. Sie kann über einen freien Port eines beliebigen Slaves

erfolgen. Allerdings sind dann nicht mehr alle Pfade redundant. Bei der Konfiguration von Bild 4.9 führt ein Kabelbruch zwischen Slave 2 und Slave 3 dazu, dass Slave 3 nicht mehr erreichbar ist. Bei einem Kabelbruch zwischen dem Master und Slave 1 bzw. zwischen Slave 1 und Slave 2 sind dagegen immer noch alle Teilnehmer erreichbar.

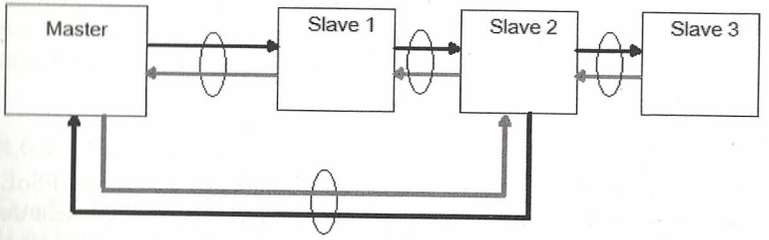


Bild 4.9: *Redundantes Teil-Netzwerk*

4.6 Safety-over-EtherCAT

Safety-over-EtherCAT ist das Protokoll für die Übertragung sicherheitsrelevanter Daten. Das Protokoll, auch ***FSoE*** (***Functional Safety over EtherCAT***) genannt, ist prinzipiell unabhängig vom Kommunikationsmedium. Entsprechend gehen Maßnahmen, die EtherCAT z. B. im Hinblick auf Datensicherung implementiert, nicht in die Berechnung der Restfehlerwahrscheinlichkeit ein. FSoE kann in Systemen eingesetzt werden, die SIL 3 verlangen.

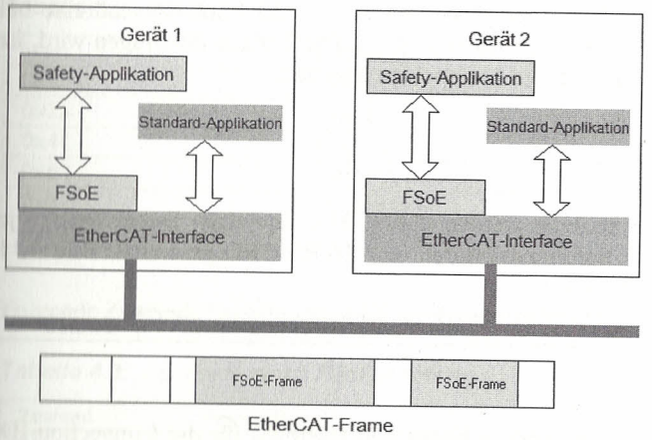


Bild 4.10: *Sichere und unsichere Daten über EtherCAT*

EtherCAT wird genutzt, um sichere und unsichere Informationen zu übertragen. FSoE-Frames werden in die EtherCAT-Prozessdaten eingebettet und in den Geräten sicher ausgewertet [BECK07b]. FSoE definiert die Rollen ***FSoE-Master*** und ***FSoE-Slave***. Ein FSoE-Master kann einen oder mehrere FSoE-Slaves verwalten. Zwischen FSoE-Master

und FSoE-Slave besteht eine ***FSoE-Connection***, die vom FSoE-Master initiiert wird. Der FSoE-Master sendet ***FSoE-Master-Frames***, die ***SafeOutputs*** enthalten. Ein FSoE-Slave beantwortet einen gültigen FSoE-Master-Frame mit einem ***FSoE-Slave-Frame***, welcher ***SafeInputs*** enthält. Danach beginnt dieser Nachrichtenzyklus von neuem.

4.6.1 Sicherheitsmaßnahmen

4.6.1.1 Watchdog

Die Zeit zwischen Absenden des FSoE-Master-Frames und Empfang eines neuen FSoE-Slave-Frames wird beim Master durch einen **Watchog** überwacht. Ebenso überwacht der Slave die Zeit zwischen dem Aussenden seines FSoE-Slave-Frames und dem Empfang eines neuen FSoE-Master-Frames durch einen Watchdog. Als Empfang eines neuen FSoE-Frames wird dabei ein gültiger FSoE-Frame gewertet, der sich in mindestens einem Bit vom zuvor empfangenen unterscheidet. Trifft innerhalb der konfigurierten FSoE-Watchdogzeit keine Antwort ein, geht der Teilnehmer in den Zustand `RESET` (s. u.).

4.6.1.2 Connection-ID, FSoE-Slaveadresse

Jede FSoE-Connection wird durch eine systemweit eindeutige 16-Bit ***Connection-ID*** identifiziert. Diese wird im Rahmen der Konfigurierung generiert und vom FSoE-Master bei der Initialisierung der FSoE-Connection an den FSoE-Slave übertragen. Da alle folgenden FSoE-Frames die Connection-ID enthalten, wird eine falsche Adressierung erkannt. Ferner besitzt jeder FSoE-Save eine systemweit eindeutige 16-Bit ***FSoE-Slaveadresse***, die z. B. über DIP-Schalter einstellbar ist. Da die FSoE-Slaveadresse bei der Initialisierung der FSoE-Connection ebenfalls an den FSoE-Slave übertragen wird, ist ein zusätzlicher Schutz gegen fehlgeleitete Nachrichten gegeben.

4.6.1.3 FSoE-Frame

Verfälschungen von FSoE-Nachrichten werden über CRCs detektiert. Dabei werden je 2 Bytes ***SafeData*** (SafeOutputs/SafeInputs) durch eine 16-Bit-CRC geschützt. Bild 4.11 zeigt den Aufbau eines FSoE-Frames.

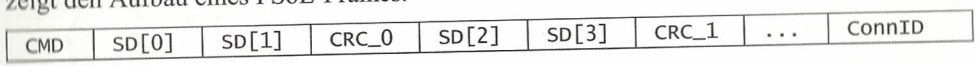


Bild 4.11: *FSoE-Frame*

Hier stehen `SD[k]` für Byte k der sicheren Daten und `ConnID` für die Connection-ID. Die Länge der sicheren Daten muss eine gerade Anzahl von Bytes oder 1 sein. Bei einem Byte SafeData entfällt `SD[1]`, und der Frame hat eine Länge von 6 Bytes. FSoE-Master-Frame und FSoE-Slave-Frame dürfen unterschiedlich lang sein.

In die `CRC_i` gehen neben den zwei Datenbytes `SD[2i]`, `SD[2i+1]` auch die Felder `CMD`, `ConnID` und die `CRC_0` des zuletzt empfangenen FSoE-Frames sowie der Index i, eine **virtuelle Sequenznummer** (16 Bit) und drei Null-Bytes ein [BECK07c]. Durch die

Einbeziehung der `CRC_0` des zuletzt empfangenene FSoE-Frames wird i. d. R. erreicht, dass sich zwei aufeinanderfolgende FSoE-Frames unterscheiden, auch wenn sich die restlichen Daten nicht verändert haben. Trotzdem kann es vorkommen, dass in zwei aufeinanderfolgenden Zyklen zwei identische Frames erzeugt werden, was zum Auslösen des Watchdogs führen würde. Aus diesem Grund wird in die CRC-Berechnung eine virtuelle Sequenznummer einbezogen, die mit jedem neuen FSoE-Frame inkrementiert wird. Unter Umständen ist es erforderlich, die virtuelle Sequenznummer mehrmals zu inkrementieren, um unterschiedliche Frames zu erhalten.

4.6.1.4 Session-ID

Durch Fehlverhalten z. B. eines Switches[3] kann es vorkommen, dass eine Hochlaufsequenz (s. u.) zwischengespeichert und zu einem falschen Zeitpunkt eingefügt wird. Indem bei Aufbau der FSoE-Connection eine zufällig erzeugte Session-ID (16 Bits) übertragen wird, ist sichergestellt, dass sich zwei Hochlaufsequenzen unterscheiden. Ferner ist durch die Einbeziehung der „alten" `CRC_0` in die CRC-Berechnung sichergestellt, dass eine Frame-Sequenz immer von der Historie abhängt.

4.6.2 Zustände eines FSoE-Knotens

Ein FSoE-Knoten (FSoE-Master oder FSoE-Slave) kann die in Tabelle 4.3 beschriebenen Zustände einnehmen. Zustandsübergänge erfolgen durch interne Ereignisse (z. B. Kommunikationsfehler) oder durch Kommandos des FSoE-Masters. Das Kommando wird im Feld `CMD` des FSoE-Frames kodiert:

Tabelle 4.2: *Werte für CMD*

Wert	Bedeutung
0x36	Process Data
0x2A	Reset
0x4E	Session
0x64	Connection
0x52	Parameter
0x08	Fail-Safe

Folgende Zustände sind für einen FSoE-Knoten definiert:

Tabelle 4.3: *Zustände eines FSoE-Knotens*

Zustand	Beschreibung
RESET	Die FSoE-Connection ist zurückgesetzt. Ausgänge im sicheren Zustand.
SESSION	Die Session-ID wird übertragen. Ausgänge im sicheren Zustand.
CONNECTION	Die Connection-ID und die FSoE-Slaveadresse werden übertragen. Ausgänge im sicheren Zustand.
PARAMETER	Die Parameter (z. B. Watchdog-Time) werden übertragen. Ausgänge im sicheren

[3] FSoE ist unabhängig vom Kommunikationssystem ausgelegt. Daher werden auch Fehler betrachtet, die von Switches verursacht werden können.

	Zustand.
DATA	Prozessdaten bzw. FailSafe-Daten werden übertragen. Ausgänge sind nur aktiv, wenn Kommando `Process Data` empfangen wurde.

Eine fehlerfreie Hochlaufphase läuft folgendermaßen ab [BECK07c]. Nach Power-On oder nach einem Kommunikationsfehler wird der Zustand `RESET` eingenommen. Die `CRC_0` des letzten Telegramms und die virtuelle Sequenznummer werden zurückgesetzt. Der FSoE-Master sendet das Kommando `Session` und geht in den Zustand `SESSION` über. Der Empfang eines korrekten `Session`-Kommandos veranlasst den FSoE-Slave, in den Zustand `SESSION` zu wechseln. Das `Session`-Kommando enthält in `SD[0]` und `SD[1]` die Session-ID. Der FSoE-Slave quittiert das Kommando, indem er ebenfalls ein `Session`-Kommando mit einer eigenen Session-ID sendet. Mit Empfang dieser Quittung geht der FSoE-Master in den Zustand `CONNECTION` über und sendet das Kommando `Connection`. Dieses enthält die Connection-ID und die FSoE-Slaveadresse. Mit Empfang eines gültigen `Connection`-Kommandos geht der FSoE-Slave in den Zustand `CONNECTION` und quittiert, indem er die sicheren Daten zurückschickt. Mit Empfang der Quittung geht der FSoE-Master in den Zustand `PARAMETER` und sendet mit Kommando `Parameter` sicherheitsrelevante Parameter wie die Watchdog-Time an den FSoE-Slave. Der FSoE-Slave quittiert, indem er die sicheren Daten zurückschickt. Die Quittung veranlasst den FSoE-Master, den Zustand `DATA` einzunehmen. Nun sendet der FSoE-Master mit den Kommandos `Process Data` bzw. `Fail-Safe` SafeOutputs. Der Empfang des ersten gültigen Kommandos dieser Art veranlasst den FSoE-Slave zum Übergang in den Zustand `DATA`. Im Fall `CMD = Process Data` werden die SafeOutputs im FSoE-Frame auf die Ausgänge geschaltet, und bei `CMD = Fail-Safe` werden die Ausgänge in den sicheren Zustand versetzt. Der FSoE-Slave antwortet mit SafeInputs. Dieser Nachrichtenzyklus wird solange wiederholt, bis entweder ein Kommunikationsfehler auftritt oder ein FSoE-Knoten gestoppt wird.

Die Kommandos `Session`, `Connection` und `Parameter` verwenden die für die sicheren Daten reservierten Felder `SD[k]` zur Übertragung ihrer Informationen (Session-ID, Connection-ID,...). Die Anzahl der Felder, die so genutzt werden können, wird durch den kürzeren der beiden Frames (FSoE-Master-Frame und FSoE-Slave-Frame) bestimmt. Die übrigen Felder im längeren Frame werden mit 0 besetzt. Es kann also vorkommen, dass für die Übertragung der Informationen mehrere Zyklen erforderlich sind. Beispielsweise benötigt man für die Übertragung der Session-ID zwei Zyklen, wenn nur ein Byte an sicheren Daten vorhanden ist.

4.7 Referenzmodell

Um den Anforderungen an die Übertragung von Prozessdaten und Bedarfsdaten gerecht zu werden, unterstützt EtherCAT eine Vielzahl von Protokollen und Diensten. Dies äußert sich in der Protokollarchitektur eines EtherCAT-Slaves.

Der ***Data Link Layer*** (***DL***) enthält die zeitkritischen Aufgaben bei der Kommunikation zwischen EtherCAT-Geräten. Er wird durch einen sog. ***EtherCAT Slave Controller*** (***ESC***) realisiert.

Oberhalb vom DL (***DL-User***) können verschiedene Protokollfamilien koexistieren. Internet-Protokolle und Echtzeitkommunikation können im selben Gerät implementiert sein.

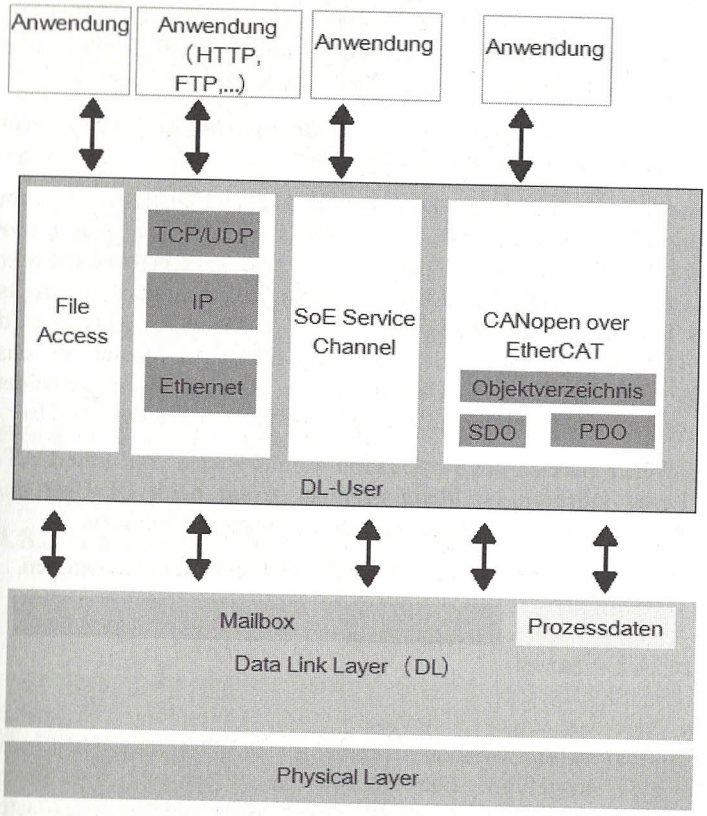


Bild 4.12: *Referenzmodell eines EtherCAT-Slave*

Die Schnittstelle zwischen DL und DL-User wird als ***Physical Device Interface*** (***PDI***) bezeichnet. Diese kann je nach Slave-Komplexität verschieden ausfallen, beispielsweise als 16-Bit DPRAM-Schnittstelle zu einem Mikrocontroller. Im einfachsten Fall handelt es sich um eine 4-Bit-I/O-Schnittstelle.

4.8 Gerätemodell

Jedes EtherCAT-Gerät muss die ***EtherCAT State Machine*** (***ESM***) und die Übertragung von **EtherCAT-Prozessdaten** unterstützen. Die ESM definiert, welche Initialisierungsschritte ein Slave durchlaufen muss, um schließlich Prozessdaten übertragen zu können. Ferner spezifiziert die ESM das Slave-Verhalten im Fehlerfall.

Jeder Slave besitzt einen 64 KB[4] großen Adressraum. Dieser kann vom EtherCAT-Master, aber auch intern angesprochen werden. Die ersten 4 KB des Adressraums sind durch **Register** belegt. Der Rest wird anwendungsspezifisch verwendet (***Application Memory***).

Register `AL Status` (Adresse `0x130`) zeigt den aktuellen Zustand der ESM. Der EtherCAT-Master fordert im Register `AL Control` (Adresse `0x120`) Zustandsänderungen an, welche der Slave in `AL Status` quittiert. Kann der Slave die Zustandsänderung nicht durchführen, so setzt er in `AL Status` ein Fehler-Flag.

Muss der Slave aufgrund einer internen Fehlersituation einen Zustandswechsel durchführen, so wird dies ebenfalls in `AL Status` angezeigt.

Greift der EtherCAT-Master **schreibend** auf ein **Register** zu, so kann dies (abhängig vom geschriebenen Register) als Ereignis an den DL-User gemeldet werden (***Write Register Event***). In diesem Fall wird der DL-User durch einen lokalen Lesezugriff den vom Master geschriebenen Registerwert auslesen. Die Event-Meldung kann in Form eines Interrupts geschehen. Im Fall eines **lesenden Registerzugriffs** durch den EtherCAT-Master wird kein Ereignis an den DL-User gemeldet. Schreibt der EtherCAT-Master in das **Application Memory**, so wird dies dem DL-User als ***Write Memory Event*** gemeldet (Interrupt). Es folgt das Auslesen des geschriebenen Speicherbereichs durch den DL-User.

Ein vom DL-User geschriebener Bereich des Application Memory wird vom EtherCAT-Master gelesen. Dieses Lesen induziert ein ***Read Memory Event*** beim DL-User des Slaves. Damit weiss der DL-User, dass er den Speicherbereich überschreiben kann.

Bei EtherCAT-Slaves wird zwischen **einfachen** und **komplexen Geräten** unterschieden.

4.8.1 Einfache EtherCAT-Slaves

Einfache Slaves verfügen über keinen eigenen Controller für die Anwendung. Der Aufbau ihrer Prozessdaten ist fest und in der Gerätebeschreibung definiert. Die ESM wird automatisch durch den ESC ausgeführt. Die Boot-Konfiguration und die Slave-Informationen befinden sich im seriellen EEPROM. Dieses kann durch den Master beschrieben werden, bevor die Schnittstelle aktiviert wird.

[4] KB: KiloByte (1024 Bytes)

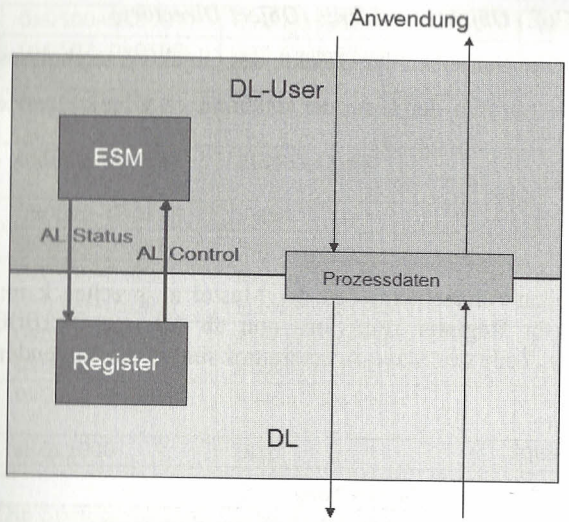


Bild 4.13: *Einfacher EtherCAT Slave*

4.8.2 Komplexe EtherCAT-Slaves

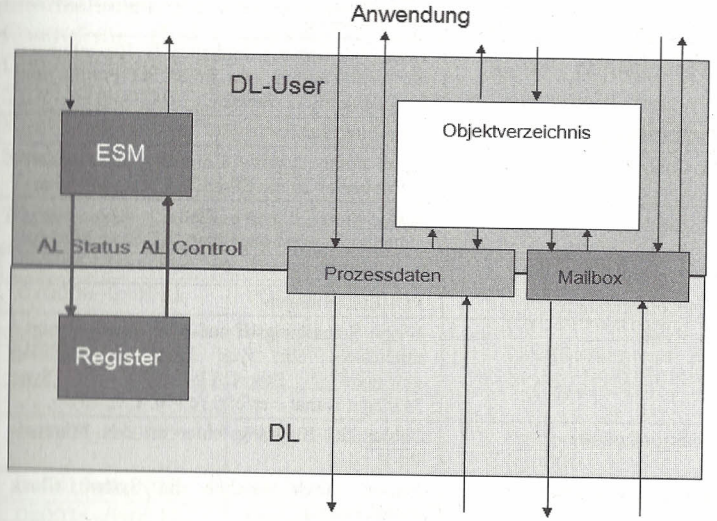


Bild 4.14: *Komplexer EtherCAT-Slave*

Komplexe Geräte verfügen über einen eigenen Controller für die Anwendung (***Application Controller***). Sie unterstützen zusätzlich:

- Eine **Mailbox**

- Das CANopen over EtherCAT (CoE) ***Objektverzeichnis (Object Directory)***
- **Expedited SDO Services** zum Lesen/Schreiben der Objekte des Objektverzeichnisses
- **SDO Information Services** zum Abrufen der Liste der unterstützen Objekte sowie deren Beschreibung.

4.9 Slave-Adressraum

Jeder Slave besitzt einen 64 KB großen Adressraum, den der Master ansprechen kann. Insbesondere sind die ersten 4 KB für **Register** reserviert, und ab Adresse 0x1000 befindet sich das Application Memory. Teile des Slave-Adressraums sind in der folgenden Tabelle zu sehen [BECK07a]:

Tabelle 4.4: *Slave-Adressraum (Auszug)*

Adresse	Parameter	Beschreibung
0x0000	Type	Typ des EtherCAT Slave Controllers
0x0001	Revision	Revision des EtherCAT Slave Controllers
0x0002-0x0003	Build	Build Number des EtherCAT Slave Controllers
0x0004	FMMU Channels	Anzahl unterstützter FMMU-Kanäle
0x0005	Sync Channels	Anzahl unterstützter Sync Manager Kanäle
0x0006	RAM Size	Größe des RAM für Prozessdaten (in KiloByte)
0x0008	FMMU Bit Operation	Bit 0 = 1: FMMU unterstützt keine Bit-Zugriffe.
0x0010-0x0011	Configured Station Address	Slave-Adresse
0x0100-0x0101	ESC DL Control	Data Link Control. Enthält z. B. Einstellungen zur Weiterleitung von EtherCAT-Frames und Nicht-EtherCAT-Frames.
0x0110-0x0111	ESC DL Status	Data Link Status
0x0120-0x0121	AL Control	Application Layer Control, insbesondere: Kommandos an die EtherCAT State Machine
0x0130-0x0131	AL Status	Application Layer Status, insbesondere: aktueller Zustand der EtherCAT State Machine
0x0140-0x0141	PDI Control	Art des PDI, ...
0x0150-0x0151	PDI Configuration	PDI-Einstellungen
0x0220-0x0223	AL Event	**Flags**: Schreibzugriff auf AL Control erfolgt / mindestens ein Sync Manager Watchdog getriggert / EtherCAT-Zugriff auf Sync Manager Kanal c erfolgt (c = 0, 1,..., 15).
0x0300-0x0307	Rx Error Counter	Zähler für Empfangsfehler an den Ethernet-Ports
0x0400-0x0401	Watchdog Divider	Faktor, durch welchen die System Clock dividiert wird
0x0410-0x0411	Watchdog Time PDI	Jeder Zugriff des Application Controller auf den Ethernet Slave Controller setzt den PDI Watchdog zurück.
0x0420-0x043F	Sync Manager Channel Watchdog	Pro Sync Manager Channel ein Watchdog
0x0450-0x0451	Sync Manager Watchdog Status	Status der Watchdogs für die Sync Manager Channels

0x0500-0x0501	Configuration	EEPROM Configuration / PDI[5] Access
0x0502-0x0503	SII[6] Control/Status	Information über Zugriff auf EEPROM und Steuerung des EEPROM-Zugriffs (bitkodiert)
0x0504-0x0507	EEPROM Address	EEPROM-Adresse, von der gelesen bzw. an die geschrieben werden soll
0x0508-0x050F	EEPROM Data	gelesene EEPROM-Daten bzw. zu schreibende EEPROM-Daten
0x0600-0x07FF	FMMU	FMMU-Kanäle
0x0800-0x8FF	Sync Manager	Sync Manager Kanäle
0x0900-0x0903	Receive Time Port 0	Empfangszeit für Port 0 (Delay-Messung)
0x0904-0x0907	Receive Time Port 1	Empfangszeit für Port 1 (Delay-Messung)
0x0908-0x090B	Receive Time Port 2	Empfangszeit für Port 2 (Delay-Messung)
0x090C-0x090F	Receive Time Port 3	Empfangszeit für Port 3 (Delay-Messung)
0x0928-0x092B	System Time Delay	Delay zwischen Reference Clock und lokaler Clock
ab 0x1000	Data	Daten, implementierungsabhängig

Die Speicherbereiche, über die der Datentransfer für Prozessdaten und Mailbox stattfindet, befinden sich im Bereich ab Adresse 0x1000 (Application Memory).

4.10 Slave Information Interface (SII)

Jeder Slave speichert seine remanenten Daten im ***Slave Information Interface*** (***SII***), welches üblicherweise als serielles EEPROM[7] realisiert ist. Das SII enthält die Informationen, die zur Initialisierung der Hardware-Schnittstelle des EtherCAT Slave Controllers (ESC) erforderlich sind. Ferner speichert es Informationen zur Geräteidentifikation (`VendorID`, `RevisionNumber`, `ProductCode`, `SerialNumber`). Die Geräteinformation ermöglicht dem Master, die zugehörige Konfiguration zu ermitteln. Die folgende Tabelle zeigt einen Ausschnitt aus der SII-Struktur.

Tabelle 4.5: *SII–Struktur (Ausschnitt)*

EEPROM-Adresse	Parameter	Beschreibung
0x0000-0x0001	PDI Control	Initialisierungswert für das PDI Control Register (0x0140-0x0141)
0x0002-0x0003	PDI Configuration	Initialisierungswert für das PDI Configuration Register (0x0150-0x0151)
...		
0x000E-0x000F	EEPROM Checksum	Prüfsumme für den SII-Bereich 0x0000-0x000D
0x0010-0x0013	VendorID	VendorID, CANopen-Objekt 0x1018:1
0x0014-0x0017	ProductCode	Product Code, CANopen-Objekt 0x1018:2
0x0018-0x001B	RevisionNumber	RevisionNumber, CANopen-Objekt 0x1018:3
0x001C-0x001F	SerialNumber	Seriennummer, CANopen-Objekt 0x1018:4

[5] PDI: Process Data Interface
[6] SII: Slave Information Interface
[7] EEPROM: Electrically Erasable Programmable Read-Only Memory

Über die **EEPROM Area** 0x500–0x50F des Slave-Adressraums kann der Master auf das Slave-EEPROM (SII) zugreifen. Der Zugriff auf das EEPROM wird über Adresse 0x500 gesteuert. Ist Bit 0 = 0, so hat EtherCAT die Kontrolle über das EEPROM (Master-Zugriff), ansonsten wird das EEPROM vom PDI kontrolliert (Slave-interner Zugriff).

Hat der Master den EEPROM-Zugriff, so kann er wie folgt auf das SII lesend oder schreibend zugreifen:

1. In Adresse 0x0502/0x0503 wird festgelegt, ob ein Lese- oder Schreibzugriff erfolgen soll.
2. Im EEPROM-Adressbereich 0x0504–0x0507 wird die Adresse hinterlegt, ab der im EEPROM gelesen bzw. geschrieben werden soll.
3. Im Fall eines Schreibzugriffs werden im Bereich 0x0508–0x0509 die Daten abgelegt, die ins EEPROM geschrieben werden sollen.
4. Bit 7 von Byte 0x0503 zeigt an, ob der EEPROM-Zugriff abgeschlossen ist.
5. Im Fall eines Lesezugriffs liegen im Adressbereich 0x0508–0x050F die aus dem EEPROM gelesenen Daten. Abhängig vom ESC werden Lesezugriffe mit 4 oder 8 Bytes unterstützt.

4.11 Adressierung

Beim Hochlauf des EtherCAT-Systems weist der Master den einzelnen Slaves **Stationsadressen** zu. Bevor dies geschehen ist, können Slaves über ihre Position im Ring angesprochen werden.

4.11.1 Physikalische Adressierung

Mit der ***physikalischen Adressierung*** wird ein Teil des Adressraums eines Slaves gelesen oder geschrieben. Es wird immer genau ein Slave angesprochen. Diese Variante kommt hauptsächlich bei der Slave-Parametrierung zum Einsatz. Zur Identifikation der einzelnen Slaves gibt es zwei Varianten:

- ***Auto-Inkrement-Adressierung (Positions-Adressierung)***: Hier wird der Slave durch seine Position im Ring identifiziert. Während des Telegrammdurchlaufs überprüft ein Slave, ob er adressiert ist, inkrementiert das 16-Bit-Adressfeld Address Page (ADP) und reicht das Telegramm zum nächsten Slave weiter. Derjenige Slave, welcher die Adresse 0 empfängt, ist adressiert. Will der Master den zehnten Slave ansprechen, so startet er mit Adressfeld -9. Der erste Slave inkrementiert den Wert auf -8 und reicht das Telegramm an den zweiten weiter. Der zweite Slave inkrementiert und übermittelt eine -7 zum dritten Slave usw. Der zehnte Slave erhält so den Wert 0. Typischerweise kommt diese Adressierungsart nur in der Hochlaufphase bei der Vergabe der Stationsadressen zur Anwendung.
- ***Fixed Address (Node Address)***: Hier wird ein Slave über seine Stationsadresse angesprochen.

4.11.2 Multicast, Broadcast

Es ist auch möglich, mit der Adressierungsart ***Multiple*** physikalische Bereiche mehrerer Slaves zu lesen. Dabei wird der erste Slave durch seine Stationsadresse adressiert. Er schaltet das `MultipleReadFlag` im Telegramm ein, über welches nachfolgende Slaves erkennen, dass sie adressiert sind. Jeder Slave, der über den spezifizierten physikalischen Adressbereich verfügt, trägt seine Stationsadresse und die entsprechenden Daten in das Datenfeld ein.

Ein ***Broadcast Write*** ermöglicht das Schreiben eines physikalischen Bereichs bei **allen** Slaves. Damit können z. B. Zustandsübergänge bei allen Slaves gleichzeitig durchgeführt werden.

4.11.3 Logische Adressierung

Der Overhead zur physikalischen Adressierung ist im Fall eines Slaves mit zwei Bit Nutzdaten nicht vertretbar. Zur Effizienzsteigerung wird daher mit einer ***Fieldbus Memory Management Unit*** (***FMMU***) gearbeitet. Diese FMMU ist im Slave-ASIC integriert. Sie übersetzt anhand einer Tabelle eine **logische Adresse** in eine **physikalische Adresse**. Dabei wird sogar bitweises Mapping unterstützt, so dass Nutzdaten, die z. B. aus zwei Bits bestehen, an eine beliebige Stelle im logischen Adressraum eingeblendet werden können.

Ein EtherCAT-Kommando zum Lesen oder Schreiben eines logischen Speicherbereichs führt dazu, dass alle Slaves, bei denen ein FMMU-Kanal eine Adressübereinstimmung feststellt, ihre Daten übernehmen bzw. ihre Daten in das EtherCAT-Datagramm einblenden. Auf diese Weise können mit einem einzelnen EtherCAT-Kommando mehrere Slaves gleichzeitig angesprochen werden.

Der logische Adressraum der FMMUs umfasst 4 GB[8]. Damit lässt sich ein EtherCAT-System aus Sicht des Masters als großer verteilter Speicher ansehen. Die Zuordnung physikalischer Adressen zu einem Slave wird im Master projektiert und in der Hochlaufphase an die FMMUs der Slaves übertragen.

Bis zu 16 FMMU-Kanäle sind konfigurierbar. Die Konfigurationsdaten für einen FMMU-Kanal belegen 16 Bytes und liegen für Kanal c ab Adresse 0x0600 + c*0x10 (c = 0, 1,..., 15) des Slave-Adressraums.

Tabelle 4.6: *Konfigurationsdaten für einen FMMU-Kanal*

Bytes	Parameter	Beschreibung
0-3	`Logical Start Address`	Logische Byte-Adresse (im globalen 4 GB-Adressraum) für die Adressübersetzung
4-5	`Length`	Länge des übersetzten Bereichs in Bytes
6-7	`LogicalStartBit/Reserved/LogicalEndBit`	Bit Offset in der logischen Startadresse/Endadresse
8-9	`PhysicalStartAddress`	Physikalische Byte-Adresse für die

[8] GB: GigaByte (1024*1024*1024 Bytes)

		Adressübersetzung im 64 KB Slave-Adressraum
10	`PhysicalStartBit/Reserved`	Bit Offset in der physikalischen Startadresse/Endadresse
11	`Type`	Enthält Flags `ReadEnable/WriteEnable`: Ist `ReadEnable = 1`, so bedeutet dies, dass eine Leseoperation (Quelle: physikalische Adresse, Ziel: logische Adresse) zugelassen ist. Ist `WriteEnable = 1`, so bedeutet dies, dass eine Schreiboperation (Quelle: logische Adresse, Ziel: physikalische Adresse) zugelassen ist.
12-15	`Enable/Reserved`	`Enable = 1` bedeutet, dass der Kanal aktiv ist

4.12 Querverkehr

EtherCAT ist als reines Master-Slave-System konzipiert. Der Master (Steuerung) erteilt Befehle an die Slaves, diese führen Befehle aus und geben Statusmeldungen zurück. Je intelligenter die Slaves, desto eher taucht das Bedürfnis auf, dass Information zwischen Slaves direkt ausgetauscht werden soll. Auch dieses Leistungsmerkmal lässt sich realisieren. Im logischen Adressraum der FMMUs werden dazu Bereiche für den Querverkehr reserviert. Der Master wechselt von Zyklus zu Zyklus zwischen einem Leseauftrag und einem Schreibauftrag für die entsprechenden Bereiche. Beim Leseauftrag blenden alle entsprechend konfigurierten Slaves ihre Daten ein, und beim Schreibauftrag entnehmen alle Slaves die für sie bestimmten Querverkehrsdaten. Der Master hat hier nur die Rolle des Verteilers.

4.13 Uhrzeitsychronisation

Bedingt durch die Kabellängen zwischen den Knoten und durch die Verzögerungszeiten, welche die einzelnen Teilnehmer verursachen, kommen Sollwerte vom Master zu unterschiedlichen Zeiten bei den Slaves an. In Anwendungen, die eine hochgenaue Synchronisation (< 10 µs) fordern, wird deshalb mit Uhrzeitsynchronisation (***Distributed Clocks, DC***) gearbeitet. Das Verfahren basiert auf IEEE 1588. Um bei allen Teilnehmern die gleiche Uhrzeit einstellen zu können, ist eine ***Reference Clock*** erforderlich, die von einem ausgewählten Slave, dem sog. ***Clock Master***, bereitgestellt wird. Der Clock Master muss im Ring vor den zu synchronisierenden Slaves liegen. Der EtherCAT-Master sendet periodisch ein Synchronisationstelegramm per Auto-Inkrement oder Fixed Address (`FRMW` oder `ARMW` (s. u.), `MultipleReadFlag = TRUE`), in welches der Clock Master seine aktuelle Zeit einträgt. Die anderen Slaves stellen ihre ***Slave Clock*** nach dieser Uhrzeit. Dazu müssen die jeweiligen Verzögerungszeiten bekannt sein.

Die Delay-Messung nutzt die Ringstruktur von EtherCAT. Mit einem Telegramm wird die Messung sämtlicher Delays veranlasst. Der Master sendet dazu einen **Broadcast Write** (`BWR`) für das Register `Receive Time Port 0` (0x0900, siehe Tabelle 4.4). Jeder Slave speichert die Empfangszeiten des Frames auf der Hin- und Rückrichtung im Register

`Receive Time Port x` (x = Port, auf dem der Frame empfangen wurde). Der Master kann nun alle Zeitstempel lesen und daraus die entsprechenden Delays berechnen. Er trägt daraufhin bei jedem Slave im Register `System Time Delay` (0x0928) den Delay zwischen Reference Clock und lokaler Clock ein. Damit kann der Slave seine Uhrzeit abgleichen.

In Bild 4.15 ist die Delay-Messung an einem einfachen Beispiel dargestellt:

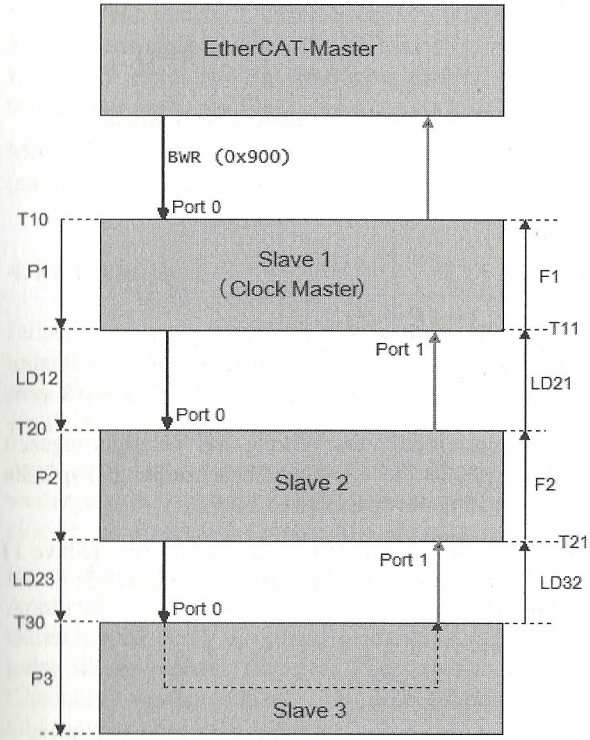


Bild 4.15: *Delay-Messung*

Dabei werden folgende Bezeichnungen verwendet [BECK07a]:

Tabelle 4.7: *Parameter für Delay-Messung*

Bezeichnung	Bedeutung	Anmerkung
T_{ij}	Zeitstempel für den Empfang des BWR-Frames am Port j von Slave i	Auf dem Hinweg wird der Frame auf Port 0 empfangen, auf dem Rückweg auf Port 1.
LD_{ik}	Line Delay von Slave i nach Slave k	Der Line Delay setzt sich aus dem Cable Delay und den jeweiligen lokalen Verzögerungen beim Senden und Empfangen zusammen (vgl. Kapitel 3). Der Line Delay wird symmetrisch angenommen ($LD_{ik} = LD_{ki}$).
P_i	Verzögerung in Slave i durch Verarbeitung des Telegramms	Für alle Slaves identisch angenommen: $P_1 = P_2 = P_3 = P$

F_i	Verzögerung in Slave i durch Weiterleitung des Telegramms (Rückweg)	Für alle Slaves identisch angenommen: $F_1 = F_2 = F$

Begonnen wird bei Slaves, die an den „Enden" liegen. Für die Zeit zwischen Empfang des BWR durch Slave 2 und Empfang durch Slave 3 ergibt sich:

$$T_{21} = T_{20} + P_2 + LD_{23} + P_3 + LD_{32} = T_{20} + 2P + 2LD_{23}$$
$$\Rightarrow P + LD_{23} = \frac{T_{21} - T_{20}}{2}. \qquad (4\text{-}1)$$

Damit lässt sich die entsprechende Zeit zwischen Slave 1 und Slave 2 bestimmen:

$$\begin{aligned} T_{11} &= T_{10} + P_1 + LD_{12} + (T_{21} - T_{20}) + F_2 + LD_{21} \\ &= T_{10} + P + 2LD_{12} + F + (T_{21} - T_{20}) \\ &= T_{10} + 2P + (F - P) + 2LD_{12} + (T_{21} - T_{20}) \\ \Rightarrow 2P + 2LD_{12} &= (T_{11} - T_{10}) - (T_{21} - T_{20}) + (P - F) \\ \Rightarrow P + LD_{12} &= \frac{(T_{11} - T_{10}) - (T_{21} - T_{20}) + (P - F)}{2}. \end{aligned} \qquad (4\text{-}2)$$

(P-F) ist die Differenz zwischen Verzögerungszeit bei Verarbeitung und Verzögerungszeit bei Weiterleitung. Diese ESC-spezifische Größe ist Teil der Gerätebeschreibung. Typische Werte sind 40 ns bei Twisted-Pair-Ports und 20 ns für E-Bus [BECK07a].

Die im Synchronisationstelegramm übertragene Uhrzeit t_1 des Clock Masters (Slave 1) führt also zu folgenden synchronisierten Uhrzeiten t_2 und t_3 bei Slave 2 und Slave 3:

Slave 2: $$t_2 = t_1 + \frac{(T_{11} - T_{10}) - (T_{21} - T_{20}) + (P - F)}{2}$$

Slave 3: $$t_3 = t_1 + \frac{(T_{11} - T_{10}) - (T_{21} - T_{20}) + (P - F)}{2} + \frac{T_{21} - T_{20}}{2}.$$

Zusätzlich zu den Delay-Messungen werden weitere Maßnahmen (Offset-Kompensation und Drift-Kompensation) zur Verbesserung der Genauigkeit implementiert.

4.14 Applikationsschicht

Standardmäßig verwendet EtherCAT das **CANopen**-Protokoll als Applikationsschicht (***CANopen over EtherCAT***, ***CoE***). Dabei ist das SDO[9]-Protokoll direkt übernommen. Auf CANopen basierende Implementierungen (insbesondere des Antriebsprofils DSP-402) lassen sich somit sehr einfach auf EtherCAT portieren. Es sind optionale Erweiterungen

[9] SDO: **S**ervice **D**ata **O**bject

definiert, die z. B. die 8-Byte-Beschränkung von SDOs und PDOs[10] aufheben, welche vom CAN[11]-Bus kommt.

Traditionell definiert eine Textdatei, die sog. EDS[12]-Datei, welche Objekte welchen Typs von einem CANopen-Gerät unterstützt werden. EtherCAT bietet hier zusätzliche Dienste in Form der sog. ***SDO Information Services***, mit deren Hilfe ein Master vom Slave die Liste der unterstützten Parameter und sämtliche Attribute der einzelnen Parameter abfragen kann.

Neben den reinen Echtzeit-Steuerungsaufgaben ist es oft wünschenswert, TCP/IP-Standardanwendungen (z. B. Webserver) in einen EtherCAT-Knoten zu integrieren. EtherCAT bietet mit ***Ethernet over EtherCAT*** (***EoE***) die Möglichkeit, Ethernet-Frames transparent durch ein EtherCAT-Netzwerk zu tunneln (siehe 4.20.5).

Mit ***SoE*** (***Servo Drive over EtherCAT***) wird ein Servicekanal für SERCOS implementiert (siehe Kapitel 2). Über diesen kann auf SERCOS-Parameter zugegriffen werden.

4.14.1 Mailbox

Jedes komplexe EtherCAT-Gerät muss über eine Mailbox verfügen. Die Mailbox unterstützt Kommunikation in beide Richtungen über mehrere Protokolle, z. B. ***CANopen over EtherCAT*** (***CoE***) oder ***Ethernet over EtherCAT*** (***EoE***). Der Mailbox Header enthält ein Feld zur Identifikation des entsprechenden Protokolls.

Azyklische Daten werden über das Mailbox-Interface an die zuständigen höheren Ebenen weitergeleitet bzw. von diesen übernommen. Zyklische Daten (Prozessdaten) werden über eine eigene **Prozessdatenschnittstelle** ausgetauscht.

Bevor die Mailbox genutzt werden kann, muss der von ihr belegte Speicherbereich des Application Memory konfiguriert werden (Startadresse, Länge). Der Zugriff auf diesen Bereich wird durch die ersten beiden ***Sync Manager Channels*** (s. u.) kontrolliert, wobei jeder dieser Kanäle für eine Übertragungsrichtung zuständig ist. Die Sync Manager Channels werden dabei im ***Handshake Mode*** (s. u.) betrieben, damit keine Überholvorgänge möglich sind.

Normalerweise richtet sich die Mailbox-Kommunikation an einen einzelnen Slave. Aus diesem Grund wird hier die physikalische Adressierung verwendet.

Ein Master, der ein Mailbox-Kommando an einen Slave sendet, muss den `WorkingCounter` (***WKC***) überprüfen, der in jedem EtherCAT-Datagramm übertragen wird (siehe Tabelle 4.18). Der `WKC` zählt die Interaktionen von Slaves, die mit einem EtherCAT-Datagramm angesprochen werden. Dazu inkrementiert ein angesprochener Slave den `WKC`, wenn auf den adressierten Speicherbereich zugegriffen werden kann. Im Fall eines erfolgreichen Schreib- und Lesezugriffs wird der `WKC` um 2 erhöht.

Der Master muss ferner zeitnah erkennen, dass ein Slave ein Mailbox-Kommando geschrieben hat, damit der Master so schnell wie möglich ein entsprechendes Lese-Kommando absetzen kann.

[10] PDO: **P**rocess **D**ata **O**bject
[11] CAN: **C**ontroller **A**rea **N**etwork
[12] EDS: **E**lectronic **D**ata **S**heet

Diese Erkennung kann auf mehrere Arten erfolgen. Eine oft genutzte Möglichkeit besteht darin, das Bit `QueuedState` (siehe Tabelle 4.11) von Sync Manager Kanal 1 mittels FMMU auf eine logische Adresse abzubilden und dieses Bit zyklisch zu lesen. Alternativ kann der Datenbereich des entsprechenden Sync Mangers gepollt werden. Ein inkrementierter `WKC` zeigt an, dass der Slave ein neues Kommando hinterlegt hat.

4.14.2 Prozessdaten

Für die Prozessdatenkommunikation wird üblicherweise ein Speicherbereich im ***buffered mode*** (s. u.) verwendet. Bei komplexen Slaves sollte der Inhalt der Prozessdaten über das **PDO Mapping** und die CANopen-Objekte „*Sync Manager PDO Assign*" festgelegt werden. Bei einfachen Slaves sind die Prozessdaten fest und sollten in der Gerätebeschreibungsdatei definiert sein. Die Prozessdatenkommunikation setzt voraus, dass die betreffenden Speicherbereiche im Application Memory konfiguriert sind. Dies wird durch die Zustandsmaschine sichergestellt.

4.15 EtherCAT State Machine

EtherCAT definiert eine Zustandsmaschine (***EtherCAT State Machine***, ***ESM***) mit fünf Zuständen sowie Kommandos, die zu Zustandsübergängen führen [IEC06c]. Die ESM ist an die Zusandsmaschine von CANopen (vgl. Kapitel 2) angelehnt. Die Zustandsübergänge werden i. d. R. vom Master veranlasst. Die einzelnen Zustände haben folgende Bedeutung:

Tabelle 4.8: *Zustände der ESM*

Zustand	`AL Status` (Bits 0-4)	**Beschreibung**
`INIT`	`0x01`	In diesem Zustand ist auf Anwendungsebene keine direkte Kommunikation zwischen Master und Slave möglich. Der Master initialisiert Konfigurationsregister des EtherCAT Slave Controllers. Verfügt der Slave über eine Mailbox, so werden die zugehörigen Sync Manager (s. u.) konfiguriert.
`PRE-OPERATIONAL`	`0x02`	Die EtherCAT Mailbox (falls vorhanden) ist aktiv. Master und Slave können über die Mailbox anwendungsspezifische Initialisierungen durchführen. Prozessdaten können nicht übertragen werden.
`SAFE-OPERATIONAL`	`0x04`	Die Ausgangsdaten befinden sich im sicheren Zustand und können nicht verändert werden. Eingangsdaten können ausgetauscht werden.
`OPERATIONAL`	`0x08`	Ausgangsdaten und Eingangsdaten können ausgetauscht werden.
`BOOTSTRAP` (optional)	`0x03`	Der Slave ist bereit, einen Firmware-Update durchzuführen.

Durch folgende Master-Kommandos werden Zustandsübergänge angefordert:

Tabelle 4.9: *Anforderung von ESM-Zustandsübergängen*

Kommando	`AL Control` **(Bits 0-3)**
Request Init State	`0x01`
Request Pre-Operational State	`0x02`
Request Safe-Operational State	`0x04`
Request Operational State	`0x08`
Request Bootstrap State	`0x03`

Folgende Aktionen sind mit den Zustandsübergängen verbunden:

Tabelle 4.10: *ESM-Zustandsübergänge*

Dienst	**Beschreibung**
Start Mailbox Communication (IP)	Der Master fordert den Übergang von `INIT` nach `PRE-OPERATIONAL` an. Zuvor muss er die zur Mailbox gehörigen Sync Channels konfigurieren. Der Slave muss ebenfalls die Konfigurierung der Sync Channels auslesen und die Mailbox entsprechend konfigurieren, bevor er den Zustandsübergang positiv quittiert.
Stop Mailbox Communication (PI)	Der Master fordert einen Übergang von `PRE-OPERATIONAL` nach `INIT` an. Der Slave beendet die Mailbox-Kommunikation, bevor er den Übergang bestätigt.
Start Input Update (PS)	Der Master fordert einen Übergang von `PRE-OPERATIONAL` nach `SAFE-OPERATIONAL` an. Zuvor muss er die Sync Manager und FMMUs für die Prozessdaten konfiguriert haben. Der Master beginnt direkt nach dem Kommando mit der Prozessdatenübertragung. Eingangsdaten vom Slave werden aber so lange ignoriert, bis der Slave den Zustandsübergang bestätigt hat. Der Slave liest die Sync Manager Konfiguration für die Prozessdaten und gleicht diese mit seiner lokalen Konfiguration ab. War diese Überprüfung erfolgreich, so überträgt der Slave sofort gültige Eingangsdaten, ehe er den Zustandsübergang bestätigt. Stellt der Slave einen Fehler bei der Überprüfung der Konfigurationsdaten fest, so bleibt er im Zustand `PRE-OPERATIONAL` und setzt den Parameter `ErrorFlag` auf `TRUE`.
Stop Input Update (SP)	Der Master fordert einen Übergang von `SAFE-OPERATIONAL` nach `PRE-OPERATIONAL` an. Zuvor hat er die Übertragung von Prozessdaten eingestellt. Der Slave stellt die Aktualisierung seiner Eingangsdaten ein, bevor er den Zustandsübergang bestätigt. Im Fall eines unerwarteten Fehlers kann die Slave-Anwendung diesen Dienst nutzen, um einen lokalen Zustandsübergang anzuzeigen.
Start Output Update (SO)	Der Master fordert einen Übergang von `SAFE-OPERATIONAL` nach `OPERATIONAL` an. Zuvor beginnt der Master mit der Übertragung gültiger Ausgangsdaten. Der Slave schaltet die Ausgangsdaten auf seine Ausgänge durch, bevor er den Zustandsübergang bestätigt. Kann der Slave die Ausgangsdaten nicht aktivieren, bleibt er im Zustand `SAFE-OPERATIONAL` und setzt den Parameter `ErrorFlag` auf `TRUE`.
Stop Output Update (OS)	Der Master fordert einen Übergang von `OPERATIONAL` nach `SAFE-OPERATIONAL` an. Der Slave versetzt seine Ausgänge in den sicheren Zustand, bevor er den Zustandsübergang bestätigt. Im Fall eines unerwarteten Fehlers kann die Slave-Anwendung diesen Dienst nutzen, um einen lokalen Zustandsübergang anzuzeigen.

Stop Output Update, Stop Input Update (OP)	OS+SP
Stop Input Update, Stop Mailbox Communication (SI)	SP+PI
Stop Output Update, Stop Input Update, Stop Mailbox Communication (OI)	OS+SP+PI
Start Bootstrap Mode (IB)	Der Master fordert einen Übergang von INIT nach BOOTSTRAP an. Zuvor muss der Master die Sync Manager für die Mailbox konfiguriert haben. Der Slave muss ebenfalls die Konfigurierung der Sync Channels auslesen und die Mailbox entsprechend konfigurieren, ehe er den Zustandsübergang positiv quittiert.
Restart Device (BI)	Neustart

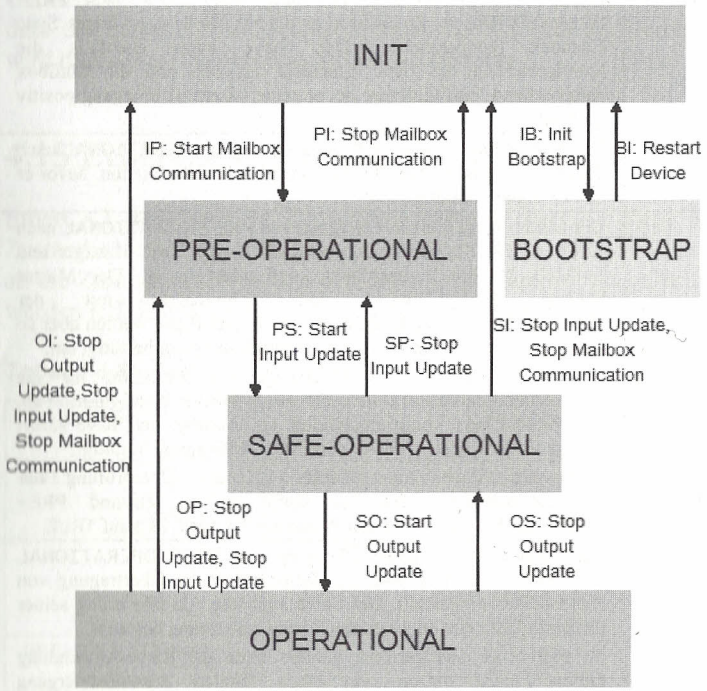


Bild 4.16: *EtherCAT State Machine (ESM)*

4.16 Sync Manager

Ein ***Sync Manager*** überwacht den Zugriff auf Anwender-Speicherbereiche. Da die lokale Update-Rate (Übernahme der Ausgangsdaten vom DL durch den DL-User bzw. Übergabe neuer Eingangsdaten durch den DL-User an den DL) nicht notwenig mit der Update-Rate über das Netzwerk übereinstimmt, müssen beide Zugriffe koordiniert werden. Diese Aufgabe übernehmen die Sync Manager. Jeder ***Sync Manager Channel*** definiert einen

konsistenten Bereich des Anwender-Speichers. Bis zu 16 unabhängige Sync Manager Channels sind möglich. Diese werden mit SM0, SM1,..., SM15 bezeichnet.

Dabei gibt es folgende ***Zugriffsarten***:

- ***Buffered Mode*** (***3-Buffer Sync Manager***): Hier stehen drei aufeinanderfolgende Speicherbereiche zur Verfügung. Jederzeit wird ein konsistenter Zugriff auf die neuesten Daten garantiert. Es gibt stets einen freien Puffer, in den geschrieben werden kann, bzw. einen konsistenten Puffer, aus dem gelesen werden kann. Bei dieser Zugriffsart kann der Anwender-Speicher jederzeit vom Kommunikationspartner geschrieben bzw. gelesen werden. Der Speicherbereich wird vom zugehörigen Sync Manager nie gesperrt.
- ***Handshake Mode*** (***Queued Mode***, 1-Buffer Mailbox Sync Manager): Hier steht nur ein Speicherbereich zur Verfügung. Ein Handshake-Mechanismus verhindert einen Überlauf. Es kann erst gelesen werden, nachdem ein Schreibvorgang abgeschlossen wurde. Vor einem erneuten Schreibzugriff muss ein Lesezugriff erfolgen.

4.16.1 Handshake Mode

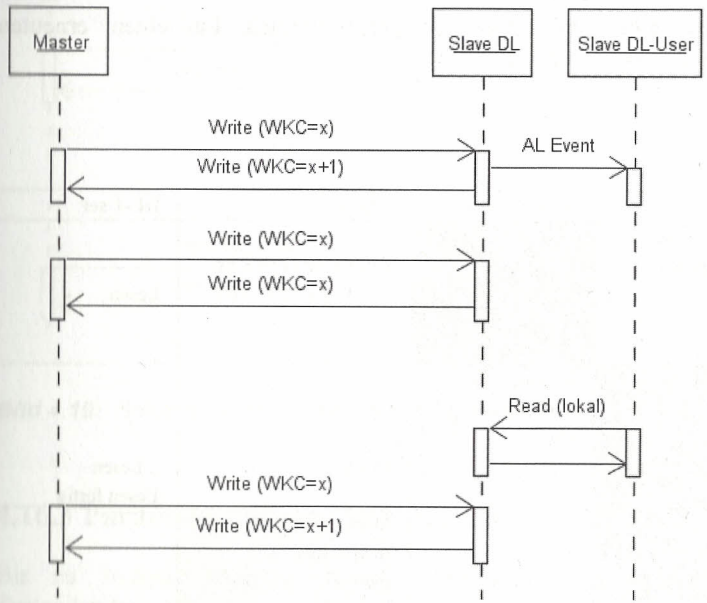


Bild 4.17: *Schreibzugriff im Handshake Mode*

Im Handshake Mode läuft ein erfolgreicher **Schreibzugriff durch den Master** wie folgt ab:

1. Der Master sendet einen Write Request mit WKC = x an den Slave.

2. Der ESC schreibt die empfangenen Daten in den betreffenden Anwender-Speicherbereich, erhöht den WKC auf x+1 und signalisiert dem DL-User ein AL Event. Der zugehörige Synch Manager sperrt den Speicherbereich, bis er gelesen wurde.
3. Der Master sieht eine positive Write Response, weil der WKC inkrementiert wurde.
4. Der DL-User liest den Anwender-Speicherbereich, und der zugehörige Sync Manager gibt den Bereich für einen erneuten Schreibzugriff durch den Master frei.

In Bild 4.17 wird der zweite Schreibzugriff des Masters abgewiesen (WKC = x), weil die zuvor geschriebenen Daten noch nicht gelesen wurden.

Ein erfolgreicher **Lesezugriff durch den Master** läuft im Handshake Mode wie folgt ab:

1. Der DL-User aktualisiert den Anwender-Speicherbereich. Der zugehörige Sync Manager sperrt den Speicherbereich, bis er vom Master gelesen wurde. Weitere Schreibzugriffe durch den DL-User werden ignoriert, solange der Master-Lesezugriff noch nicht erfolgt ist.
2. Der Master sendet einen Read Request mit WKC = x. Der ESC sendet die Daten des Anwender-Speicherbereichs, inkrementiert den WKC und signalisiert den Lesezugriff durch ein AL Event an den DL-User.
3. Der Master sieht eine positive Read Response, weil der WKC inkrementiert wurde.
4. Der Sync Manager gibt den Anwender-Speicherbereich für einen erneuten Schreibzugriff durch den DL-User frei.

4.16.2 Buffered Mode

EtherCAT	ECAT	NEXT	USER	DL-User
Schreiben... Schreiben fertig	wwwwwwww wwwwwwww ww		rrrrrrrrrrrrrr rrrrrrrrrrrrrr	Lesen
		wwwwwwww wwwwwwww ww	rrrrr	...Lesen Lesen fertig
Schreiben...	wwwww		wwwwwwww wwwwwwww ww	

Bild 4.18: *Pufferwechsel im Buffered Mode*

Beim **Buffered Mode** ist dem Sender und dem Empfänger jeweils ein Puffer zugewiesen. Ein dritter Puffer dient als Zwischenspeicher. Jede Seite sieht nur den ihr zugewiesenen Puffer. Wurde das letzte Byte geschrieben bzw. gelesen, so erfolgt automatisch ein Pufferwechsel. In Bild 4.18 sind in der Spalte „EtherCAT" Schreibzugriffe des Masters

dargestellt. In der Spalte „DL-User" ist das lokale Lesen im Slave zu sehen. Die Spalten „ECAT" und „USER" bezeichnen die Puffer, die dem EtherCAT-Zugriff (ECAT) und dem Lesen durch den Slave (USER) zugewiesen sind. „NEXT" kennzeichnet den Zwischenpuffer. Zu Beginn schreibt der Master, während der Slave Daten liest. Mit Beendigung des Schreibvorgangs durch den Master findet automatisch ein Pufferwechsel statt, so dass für den nächsten Schreibzugriff ein freier Puffer zur Verfügung steht. Nach Abschluss des Lesevorgangs durch den DL-User findet ebenfalls ein Pufferwechsel statt, so dass der Slave beim nächsten Lesevorgang auf die neuesten Daten zugreift. Bild 4.19 zeigt einen Schreibzugriff im Buffered Mode mit einem Master, der die Daten schneller aktualisiert, als sie vom Slave gelesen werden. Hier wird der zweite Schreibzugriff des Masters positiv quittiert, obwohl die zuvor geschriebenen Daten noch nicht gelesen wurden. Analog verhält es sich, wenn der DL-User mehrmals akualisiert, bevor der Master liest.

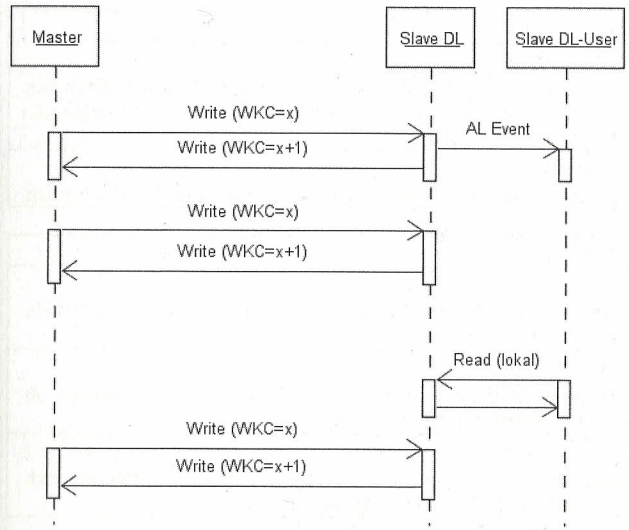


Bild 4.19: *Schreibzugriff im Buffered Mode*

4.16.3 Parameter eines Sync Channel

Bis zu 16 Sync Manager Kanäle sind konfigurierbar. Die Konfigurations- und Zustandsinformationen für einen SM[13]-Kanal belegen 8 Bytes und liegen für Kanal c ab Adresse 0x0800 + c*8 (c = 0, 1,..., 15) des Slave-Adressraums.

[13] SM: Sync Manager

Tabelle 4.11: SM-Parameter

Bytes	Beschreibung
0-1	`PhysicalStartAddress` Physikalische Startadresse (im 64 KB Slave-Adressraum) des konsistenten Speicherbereichs
2-3	`Length` Länge des Speicherbereichs in Byte
4	**Bits** / **Beschreibung** 0-1: `OperationMode` Wert 0x00: Buffered Wert 0x02: Queued (Handshake Mode) 2-3: `Direction` Wert 0x00: Lesen Wert 0x01: Schreiben 5: `ALEventEnable` Dieses Flag gibt an, ob im Fall eines erfolgten Schreibzugriffs durch den Master bzw. im Fall eines erfolgten Lesezugriffs durch den Master ein AL `Event` signalisiert werden soll. 6: `WatchdogEnable` Dieses Flag gibt an, ob der Zugriff auf den konsistenten Speicherbereich
5	**Bits** / **Beschreibung** 0: `WriteEvent` Falls `ALEventEnable` = `TRUE`, wird hier das Ereignis angezeigt, ob ein Schreibzugriff durch den Master erfolgt ist. 1: `ReadEvent` Falls `ALEventEnable = TRUE`, wird hier das Ereignis angezeigt, ob ein Lesezugriff durch den Master erfolgt ist. 2: `WatchdogTrigger` Hier wird angezeigt, ob ein Zugriff auf den konsistenten Speicherbereich erfolgt ist und ob der Watchdog ausgelöst wurde. 3: `QueuedState` Im Fall von `OperationMode` = `queued` wird hier der Zustand angezeigt: Puffer gelesen (0) / Puffer geschrieben (1). 4-5: `BufferedState` Im Fall von `OperationMode = buffered` wird hier der Zustand angezeigt: Wert 0x00: Erster Puffer Wert 0x01: Zweiter Puffer
6	**Bits** / **Beschreibung** 0: `ChannelEnable` Wert 0x00: Channel disabled Wert 0x01: Channel enabled 1: `Repeat`

	Eine Änderung dieses Parameters fordert eine Wiederholung der letzten Mailbox-Interaktionen an.
7	**Bits** / **Beschreibung** 0: `ChannelEnablePDI` Wert 0x00: Channel disabled Wert 0x01: Channel enabled 1: `RepeatAck` Eine Änderung dieses Parameters quittiert eine Anforderung zur Wiederholung der letzten Mailbox-Interaktionen.

4.16.4 CANopen-Objekte für Sync Manager

Eine Reihe von Objekten des CANopen-Objektverzeichnisses sind für Sync Manager zuständig:

Tabelle 4.12: Sync Manager CANopen-Objekte

ObjectID	Beschreibung
0x1C00	**Sync Manager Communication Type** Die Subindizes 1-32 definieren den **Kommunikationstyp** für den entsprechenden Sync Manager Channel: 0: Nicht benutzt 1: Mailbox Receive (Master → Slave) 2: Mailbox Send (Slave → Master) 3: Process Data Output (Master → Slave) 4: Process Data Input (Slave → Master)
0x1C10-0x1C2F	**Sync Manager PDO Assign** Enthält eine Liste aller PDOs, die zu einem Sync Manager Channel gehören. Die Identifikation der PDOs erfolgt durch die ObjectIDs der zugehörigen PDO Mapping Objekte.

Je nach Slave-Komplexität kann das PDO Mapping nur teilweise oder aber gar nicht unterstützt werden. So kann ein Gerät über feste Prozessdaten verfügen oder nur bestimmte vorkonfigurierte PDOs zulassen.

Es gilt folgende **Standardzuordnung** der Sync Manager Channels: Bei Geräten **ohne Mailbox** ist SM0 für Outputs (falls keine Outputs vorhanden sind, für Inputs) reserviert. SM1 ist in diesem Fall für Inputs reserviert. Bei Geräten **mit Mailbox** sind SM0 für Mailbox Receive (Master → Slave), SM1 für Mailbox Send, SM2 für Process Data Output (Master → Slave) und SM3 für Process Data Input definiert.

4.17 Status-LEDs

Ein EtherCAT-Slave zeigt i. d. R. mittels LEDs seinen Status an. Bei komplexen Slaves sind folgende LEDs spezifiziert [ETG05a]:

Tabelle 4.13: LEDs eines EtherCAT Slave

LED	Beschreibung	
Link/Activity	Diese **grüne** LED zeigt den Zustand der physikalischen Verbindung und die Aktivität auf der Verbindung. Besteht eine Verbindung, findet aber keine Übertragung statt, ist die LED an. Bei Aktivität blinkt die LED.	
RUN	Diese **grüne** LED zeigt den Zustand der ESM.	
	LED-Anzeige	ESM-Zustand
	Aus	`INIT`
	Blinken (2,5 Hz an/aus)	`PRE-OPERATIONAL`
	Periodisch 200 ms an, 1000 ms aus	`SAFE-OPERATIONAL`
	An	`OPERATIONAL`
	Blinken (10 Hz An/Aus)	Initialisierung/`BOOTSTRAP`
ERR	Diese **rote** LED zeigt Fehlerzustände an. Liegen gleichzeitig mehrere Fehlerbedingungen vor, so wird die mit der größten LED-Aktivität angezeigt.	
	LED-Anzeige	Fehlerzustand
	Aus	Kein Fehler
	Blinken (10 Hz an/aus)	Booting Error, z. B. Flash Checksum Error
	Periodisch 200 ms an, 1000 ms aus	Gerät hat selbstständig einen Zustandswechsel durchgeführt.
	Periodisch 200 ms an, dann 200 ms aus, dann 200 ms an, dann 200 ms aus, dann 1000 ms aus	Application Watchdog Timeout
	An	PDI Watchdog Timeout
STATUS	Die STATUS-LED kombiniert die RUN LED und die ERR LED in einer zweifarbigen LED.	

4.18 Gerätebeschreibung

Die Gerätebeschreibung eines EtherCAT-Slaves wird in Form einer XML-Datei vom Hersteller des Geräts bereitgestellt. Die XML-Datei basiert auf drei XML-Schemas, die von der ETG stammen [ETG08a]:

- **EtherCATInfo.xsd**: Definiert die Pflicht-Elemente der Gerätebeschreibung
- **EtherCATBase.xsd**: Definiert Datentypen
- **EtherCATDict.xsd**: Beschreibt das Objektverzeichnis eines Geräts.

Bestimmte Elemente und Attribute sind als Pflichtangaben (Mandatory) bzw. optionale Angaben klassifiziert. In einer Schema-Datei sind Pflicht-Attribute mit `use="required"` gekennzeichnet.

Im Folgenden betrachten wir einige wichtige Elemente der Gerätebeschreibung.

4.18.1 EtherCATInfo.xsd

Der Typ `EtherCATInfo` setzt sich aus den Elementen `Vendor` und `Description` zusammen:

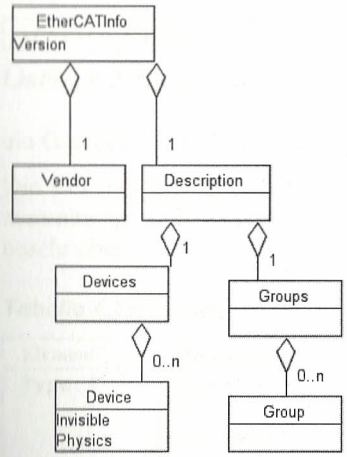


Bild 4.20: EtherCATInfo

Das `EtherCATInfo`-Attribut `Version` enthält die Version des Schemas.

4.18.1.1 Vendor

Das Element `Vendor` identifziert den Hersteller durch eine Zeichenkette, eine **ETG Vendor ID** und evtl. durch eine Bitmap (`16x14`, 16 Farben), die direkt durch hexadezimal kodierte Binärdaten angegeben oder durch einen Pfadnamen identifiziert wird.

```
<Vendor>
  <Id>68</Id>
  <Name>Hilscher GmbH</Name>
  <ImageData16x14>
424D3603000000000000360000002800000010000000100000000100180000000000000
...
  </ImageData16x14>
</Vendor>
```

Listing 4.1: Element Vendor mit eingebetteter Bitmap

Listing 4.1 zeigt das entsprechende Element für einen Beispiel-Slave auf netX-Basis (**Hilscher NXSB 100.xml**).

4.18.1.2 Description

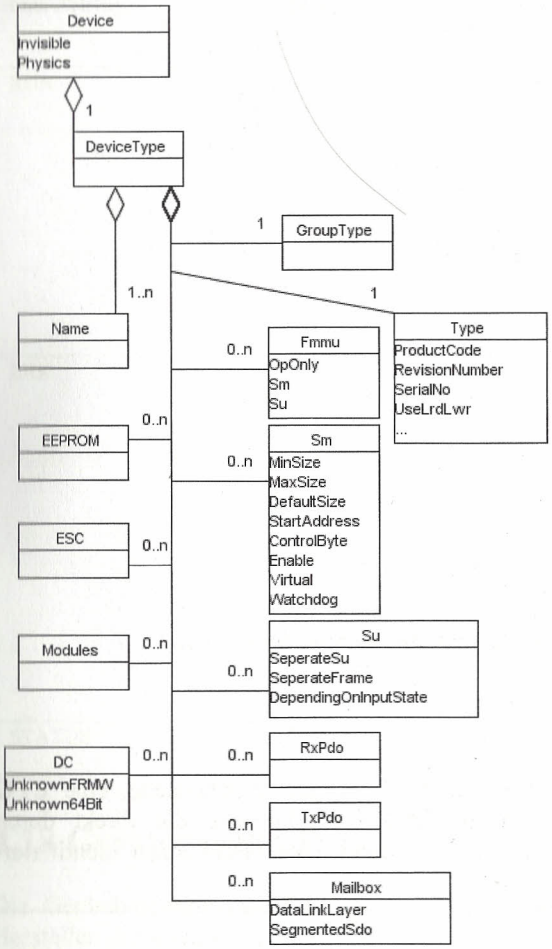


Bild 4.21: *DeviceType (vereinfacht)*

`Description` enthält die Elemente `Devices` und `Groups`. In `Groups` werden Geräte gleichen Typs zusammengefasst, die sich in bestimmten Leistungsmerkmalen unterscheiden können. `Devices` kann beliebig viele Elemente vom Typ `Device` (Bild 4.21) enthalten. Ein `Device` beschreibt sämtliche Eigenschaften und Parameter eines Geräts. Das `Device`-Attribut `Physics` gibt für jeden Ethernet-Port in Form eines Buchstabens das Medium an. Der erste Buchstabe bezieht sich auf Port 0, der zweite auf Port 1 usw.

Tabelle 4.14: *Werte für Attribut Physics*

Wert	Bedeutung
Y	Twisted-Pair-Kabel (100Base-TX)
F	Ethernet LWL (100Base-FX)
B	E-Bus
K	E-Bus Backplane

Beispielsweise kennzeichnet

```
<Device Physics="YY">
```

Listing 4.2: *Device-Attribut Physics*

ein Gerät mit zwei Twisted-Pair-Ports (Port 0 und Port 1).

Die weiteren Eigenschaften eines `Device` werden im Element vom Typ `DeviceType` zusammengestellt. Einige Elemente von `DeviceType` werden in Tabelle 4.15 beschrieben.

Tabelle 4.15: *Elemente von DeviceType*

Element	Beschreibung	Beispiel
`Type`	Herstellerspezifische Typbezeichnung. Das Attribut `UserLrdLwr` wird nicht mehr benutzt.	`<Type ProductCode="#x7874656E" RevisionNo="#x00030001" UserLrdLwr="1">NXSB100</Type>`
`Name`	Gerätename. Attribut `LcId` gibt die Sprache des Elements an (MS Windows Kodierung). 1033 steht z. B. für English (Great Britain). ***Anmerkung:*** Mit „`<![CDATA [`“ und mit „`]]>`“ wird ein sog. ***CDATA[14]-Abschnitt*** *definiert. CDATA ist ein Begriff aus der SGML[15]. Damit werden Textblöcke geschützt, die Zeichen enthalten, welche normalerweise als Markup interpretiert würden.*	`<Name LcId="1033"> <![CDATA [NXSB100 Starter Board I/O Eval Board]]></Name>`
`GroupType`	Stellt den Bezug zum `Group`-Element der Datei her.	`<GroupType>netX</GroupType>`
`Fmmu`	FMMU-Abbildungsrichtung. Attribut `Sm` definiert den zugehörigen Sync Manager. Attribut `Su` ist für künftige Verwendung vorgesehen. `Outputs` – Master → Slave `Inputs` – Slave → Master `MboxState` – Der Mailbox-Zustand wird in die Prozessdaten abgebildet.	`<Fmmu Su="0" Sm="2"> Outputs </Fmmu> <Fmmu Su="1" Sm="3"> Inputs</Fmmu> <Fmmu>MboxState</Fmmu>`
`Sm`	Default-Verwendung der Sync Manager. Attribut `ControlByte` enthält den Initialwert für das entsprechenden Sync Manager Register (0x0804+8*Kanalnummer). Die Sync	`<Sm MinSize="128" MaxSize="128" DefaultSize="128" StartAddress="#x1000"`

[14] CDATA : Character Data, Zeichendaten, Begriff aus der SGML
[15] SGML: Standard Generalized Markup Language

	Manager müssen in der Reihenfolge SM0, SM1, SM2, ... angegeben werden.
	Interpretation von `ControlByte`, siehe Tabelle 4.11. Für die Werte in der Spalte „Beispiel“:
	Wert / **Bedeutung**:
	0x26 – Queued, Schreiben `ALEventEnable` = TRUE, `WatchdogEnable` = FALSE
	0x22 – Queued, Lesen `ALEventEnable` = TRUE, `WatchdogEnable` = FALSE
	0x64 – Buffered, Schreiben `ALEventEnable` = TRUE, `WatchdogEnable` = TRUE
	0x20 – Buffered, Lesen `ALEventEnable` = TRUE, `WatchdogEnable` = FALSE
		`ControlByte="#x26"`
		`Enable="1">MBoxOut`
		`</Sm>`
		`<Sm MinSize="128"`
		`MaxSize="128"`
		`DefaultSize="128"`
		`StartAddress="#x1100"`
		`ControlByte="#x22"`
		`Enable="1">MBoxIn`
		`</Sm>`
		`<Sm MinSize="4" MaxSize="4"`
		`DefaultSize="4"`
		`StartAddress="#x1200"`
		`ControlByte="#x64"`
		`Enable="1">Outputs`
		`</Sm>`
		`<Sm MinSize="4" MaxSize="4"`
		`DefaultSize="4"`
		`StartAddress="#x1280"`
		`ControlByte="#x20"`
		`Enable="1">Inputs`
		`</Sm>`

`RxPdo`
`TxPdo`
	Receive PDOs / Transmit PDOs des Geräts. Hat Attribut `Fixed` den Wert 1, so ist die PDO nicht konfigurierbar. Unter `Index` wird der zugehörige Mapping-Eintrag abgelegt. `Sm` verweist auf den Sync Manager Channel.	`<RxPdo Fixed="1" Sm="2" Su="0">`
		`<Index>#x1600</Index>`
		`<Name>wOut0</Name>`
		`<Entry>`
		`<Index>#x2003</Index>`
		`<SubIndex>7</SubIndex>`
		`<BitLen>32</BitLen>`
		`<Name>Output</Name>`
		`<DataType>DINT</DataType>`
		`</Entry>`
		`</RxPdo>`
		
		`<TxPdo Fixed="1" Sm="3" Su="1">`
		`<Index>#x1a00</Index>`
		`<Name>wIn0</Name>`
		`<Entry>`
		`<Index>#x2003</Index>`
		`<SubIndex>4</SubIndex>`
		`<BitLen>32</BitLen>`
		`<Name>Input</Name>`
		`<DataType>DINT</DataType>`
		`</Entry>`
		`</TxPdo>`

`Mailbox`	Beschreibung der Mailbox-Protokolle.	`<Mailbox>`

	Ein vorhandenes Element `CoE`, `FoE`, `EoE`, `VoE`, `SoE` zeigt an, dass das entsprechende Mailbox-Protokoll unterstützt wird. Innerhalb dieses Elements können weitere Attribute spezifiziert werden. Bei `CoE` gibt das Attribut `PdoConfig` beispielsweise an, ob das PDO Mapping des Geräts veränderbar ist. Ferner kann mit Attribut `EdsFile` der Pfad für eine EDS-Datei angegeben werden, aus der die unterstützten CANopen-Objekte entnommen werden sollen. Dieses EDS-Objektverzeichnis soll anstelle des Online-Objektverzeichnisses verwendet werden. Attribut `SdoInfo` spezifiziert, ob Schreibzugrif auf die SDO Information besteht.	`<CoE SdoInfo="1"/>` `</Mailbox>`

DC	Distributed Clocks. Attribut `UnknownFRMW` spezifiziert, ob das Gerät `FRMW` unterstützt (1 = Nein). Die Unterstützung von 64-Bit-Zeitwerten wird durch `Unknown64Bit` definiert (1 = 64 Bit unterstützt). In den Elementen von `OpMode` wird u. a. die Zykluszeit für die Synchronisation festgelegt.	`<Dc UnknownFRMW ="1"`
		`Unknown64Bit="0">`
		`<OpMode>`
		`<Name>DcSync</Name>`
		`<Desc>DC`
		`for synchronization`
		`</Desc>`
		`<AssignActivate>#x0300`
		`</AssignActivate>`
		`<CycleTimeSync0`
		`Factor="1">0`
		`</CycleTimeSync0>`
		`<ShiftTimeSync0>0`
		`</ShiftTimeSync0>`
		`</OpMode>`
		`</Dc>`

4.19 TwinCAT und EtherCAT

In diesem Abschnitt wird die Einbindung eines EtherCAT-Slaves in ein Netzwerk behandelt. Dazu sind ein EtherCAT-Master und ein Konfigurationswerkzeug erforderlich. Beides ist in dem Software-System **TwinCAT** enthalten. TwinCAT ist eine offene PC-Softwarelösung für SPS und NC von der Firma Beckhoff. Zum Test kann eine auf 30 Tage begrenzte TwinCAT-Vollversion von http://www.beckhoff.de heruntergeladen werden. Der PC kann zur Entwicklung von Steuerungsprogrammen, aber auch zu deren Ausführung genutzt werden. TwinCAT setzt dabei eine von Beckhoff entwickelte Windows-Echtzeiterweiterung ein.

TwinCAT enthält einen EtherCAT-Master. Dieser kann über eine Ethernet-Schnittstelle des PC kommunizieren. Für harte Echtzeit benötigt man hierbei allerdings einen auf Intel 8255x basierenden Netzwerkadapter. Verzichtet man auf diese Anforderung, so kann der EtherCAT-Master über jede Ethernet-Schnittstelle betrieben werden.

Als EtherCAT-Slave können beispielsweise ein netX-Starterboard (NXHX) oder ein netSTICK verwendet werden. Wie die beiden Geräte als einfacher EtherCAT-Slave

betrieben werden, ist im Anhang (A.2) beschrieben. Die niederwertigen 4 bzw. 6 Bits der Ausgangsdaten vom EtherCAT-Master werden auf dem netX-Gerät mit LEDs angezeigt. Im Fall des NXHX kann das niederwertige Byte der Eingangsdaten über DIP-Schalter verändert werden. Beim netSTICK werden die Eingangsdaten alle 10 Sekunden inkrementiert. Zwei weitere LEDs werden als EtherCAT-Status-LEDs ERR und RUN betrieben. Folgende Gerätebeschreibungen sind hierfür zu verwenden:

Tabelle 4.16: *Gerätebeschreibungen für netX-Geräte*

Gerät	Gerätebeschreibung
NXHX	**Hilscher NXSB 100.xml** (siehe A.2.1)
netSTICK	**Hilscher netSTICK.xml** (siehe A.2.2)

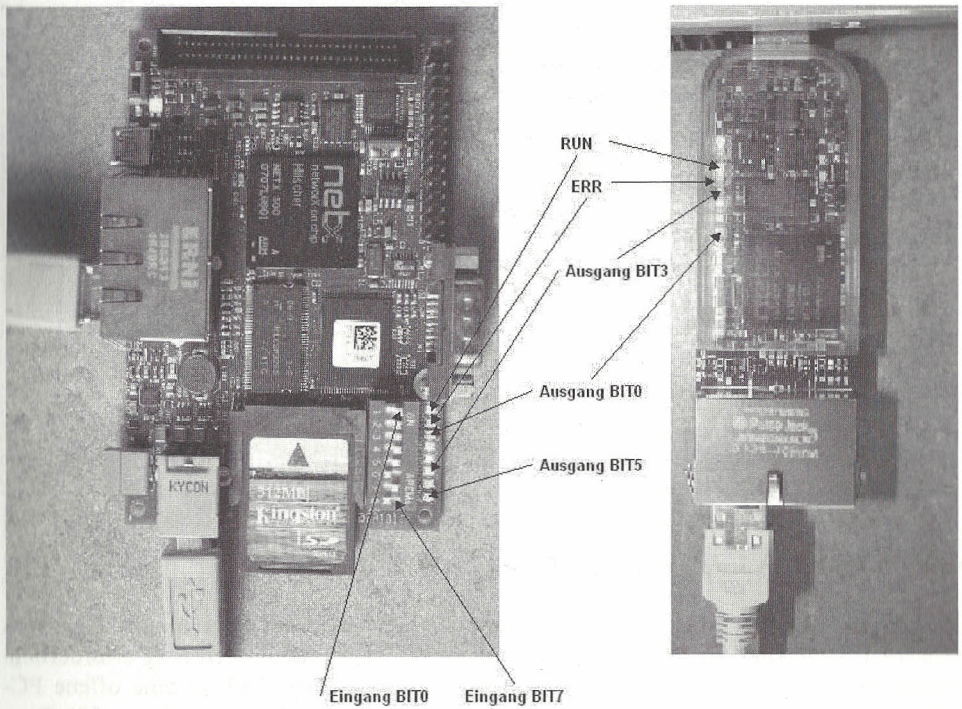


Bild 4.22: *NXHX (links) und netSTICK (rechts) als EtherCAT-Slave*

4.19.1 TwinCAT System Manager

Der **TwinCAT System Manager** ist das zentrale Konfigurationswerkzeug von TwinCAT. Um eine Kommunikation zwischen dem EtherCAT-Master und dem EtherCAT-Slave aufzubauen, sind folgende Schritte auszuführen:

1. **Schritt:** Gerätebeschreibung für den EtherCAT-Slave in TwinCAT importieren: Datei aus Tabelle 4.16 in das Unterverzeichnis **Io\EtherCAT** des TwinCAT-

Installationsverzeichnisses kopieren.

2. **Schritt:** Im **TwinCAT System Manager** muss die Ethernet-Schnittstelle ausgewählt werden, über die der EtherCAT-Master kommunizieren soll. Dabei werden für die harte Echtzeit-Kommunikation (RT EtherCAT) nur auf Intel 8255x basierende Netzwerkadapter unterstützt. Mit **Optionen->Liste Echtzeit Ethernet kompatible Geräte...** werden die verfügbaren Adapter angezeigt. Für den RT-EtherCAT-Master kann nur ein Adapter ausgewählt werden, der unter **Compatible Devices** erscheint.

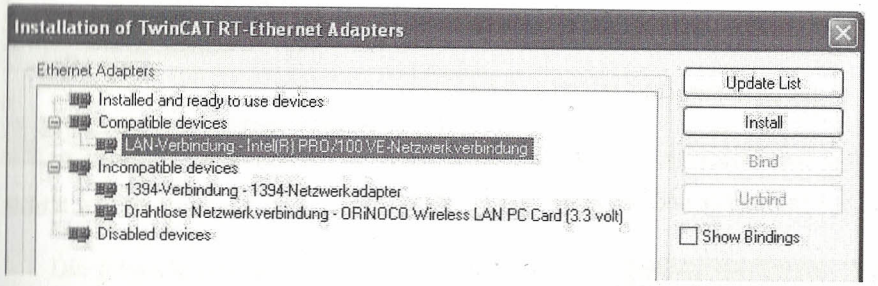


Bild 4.23: *Ethernet Schnittstelle konfigurieren*

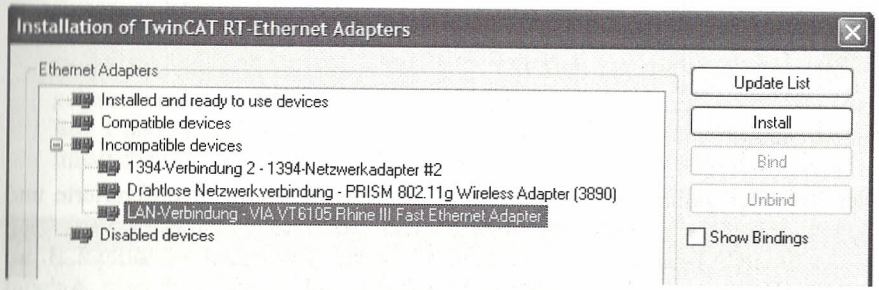


Bild 4.24: *Kein RT Ethernet kompatibles Gerät vorhanden*

Verzichtet man auf harte Echtzeit, so kann auch ein Adapter aus der Rubrik **Incompatible Devices** gewählt werden (Bild 4.24).

Man wähle nun einen Ethernet-Adapter und klicke auf die Schaltfläche **Install**, um den Standard-Treiber für diesen Adapter mit dem Echtzeit-Ethernet Miniport-Treiber bzw. einem Intermediate-Treiber von TwinCAT zu ersetzen. Es folgen evtl. mehrere Warnungen („*Windows-Logo-Test nicht bestanden*"), die man mit Anklicken von **Installation fortsetzen** übergeht. Nach erfolgreicher Treiber-Installation erscheint der Adapter unter **Installed and ready to use devices**.

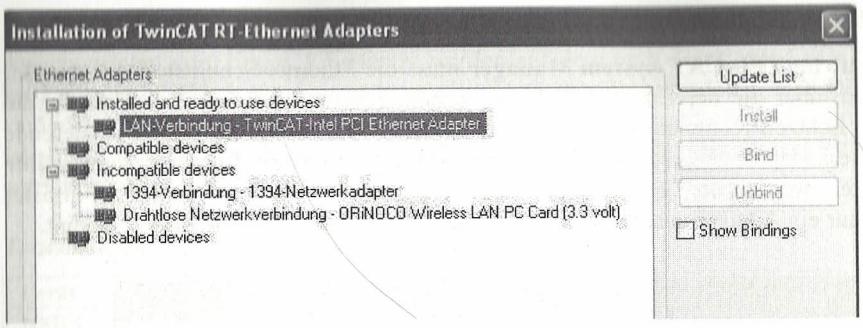


Bild 4.25: *Installierter TwinCAT RT-Ethernet Adapter*

3. **Schritt:** Unter **E/A-Konfiguration** aktiviere man das Kontextmenü des Eintrags **E/A Geräte** und wähle **Geräte Suchen...**

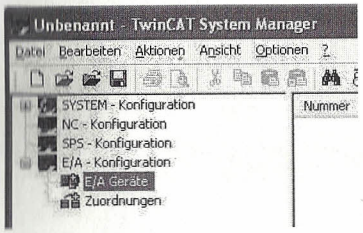


Bild 4.26: *E/A Geräte*

Aus der Liste der gefundenen Geräte wähle man das in Schritt 2 konfigurierte und bestätige mit **OK**. In der folgenden Abfrage **Nach neuen Boxen suchen** wähle man **Ja**. Ist der EtherCAT-Slave mit dem EtherCAT-Master verbunden, so sollte z. B. im Fall des NXHX ein neuer Eintrag **Box 1 (NXSB100)** in der Liste erscheinen. Anhand der Informationen `ProductCode/RevisionNumber`, welche aus dem Gerät gelesen werden, ordnet TwinCAT die Gerätebeschreibungsdatei zu. Aus dieser stammt das Bitmap-Logo, mit welchem der EtherCAT-Slave dargestellt wird (Bild 4.27).

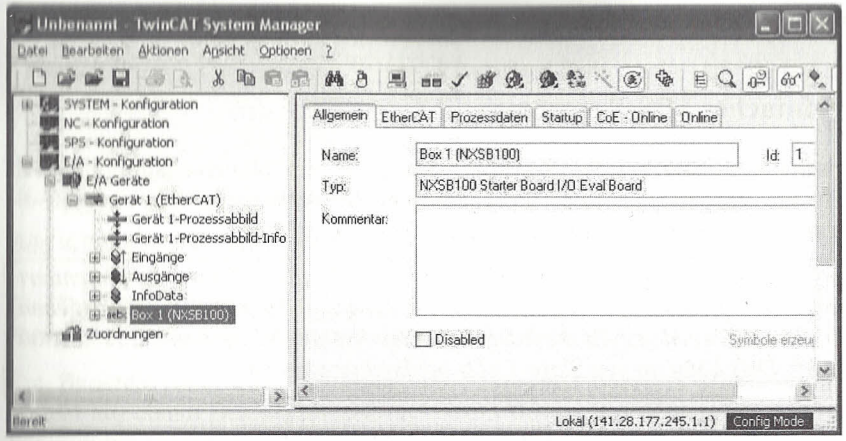


Bild 4.27: *Gefundener EtherCAT-Slave*

Die folgende Abfrage **Aktiviere Free Run** bestätige man mit **Ja**. Im **Free Run** führt der Master den Slave in den Zustand `OPERATIONAL` über, was den Zugriff auf Ein- und Ausgangsdaten der Box direkt aus dem **TwinCAT System Manager** ermöglicht. Normalerweise ist eine Applikation in Form eines SPS-Programms der Produzent der Ausgangsdaten und Konsument der Eingangsdaten (s. u.). Zu Testzwecken kann aber auch über den **TwinCAT System Manager** auf die Prozessdaten zugegriffen werden. Durch Erweitern des Eintrags **Box 1 (NXSB100)** können nun dessen Eingangsdaten gelesen und Ausgangsdaten geschrieben werden. Dazu erweitere man den Eintrag **Box 1** und klicke unter **wIn0** auf **Input**. In der Registerkarte **Online** kann der aktuelle Inhalt der Eingangsdaten verfolgt werden:

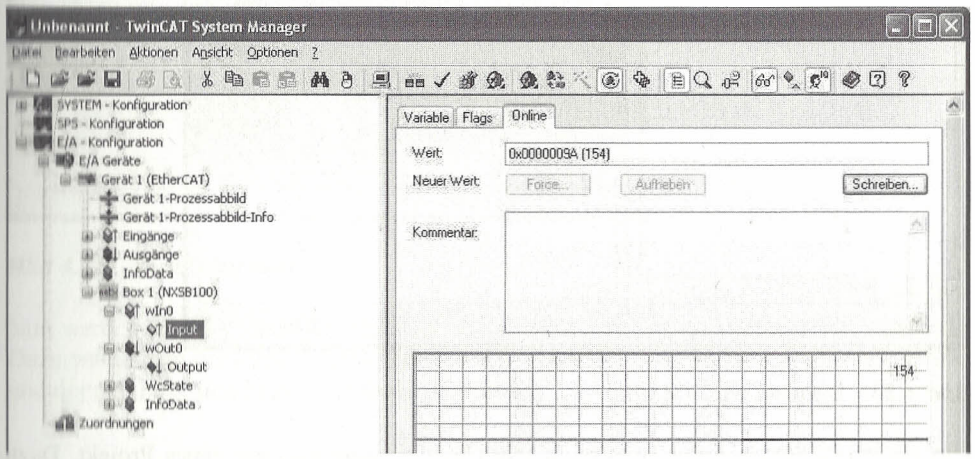


Bild 4.28: *Eingangsdaten lesen*

Wird beim NXHX die Stellung der DIP-Schalter verändert, so ändern sich die Eingangsdaten. Beim netSTICK wird der Wert der Eingangsdaten alle 10 Sekunden erhöht. Die Ausgangsdaten lassen sich über die entsprechende Registerkarte ändern.

Die aktuelle Konfiguration wird unter unter **Bsp1.tsm** gespeichert.

4.19.2 Ein einfaches SPS-Programm

Um die typische Einbindung eines EtherCAT-Slaves in ein Steuerungsprogramm zu demonstrieren, soll ein einfaches SPS[16]-Programm erstellt werden, mit dem man auf die Prozessdaten zugreift. Das Programm soll folgende Aufgabe erfüllen:

> *Ein Ausgangs-Datenbyte, das an den Slave geht, soll alle 4 *n ms inkrementiert werden. Dabei ist n die Zahl, deren Bits durch die Eingangsdaten des Slaves gegeben sind. Im Fall n = 250 werden z. B. die Ausgangsdaten in Intervallen von 4*250 = 1000 ms hochgezählt. Dies kann an den Slave-LEDs beobachtet werden.*

4.19.2.1 Programmerstellung: TwinCAT PLC Control

Zur Programmerstellung startet man **TwinCAT PLC Control**, die Entwicklungsumgebung für Steuerungsprogramme:

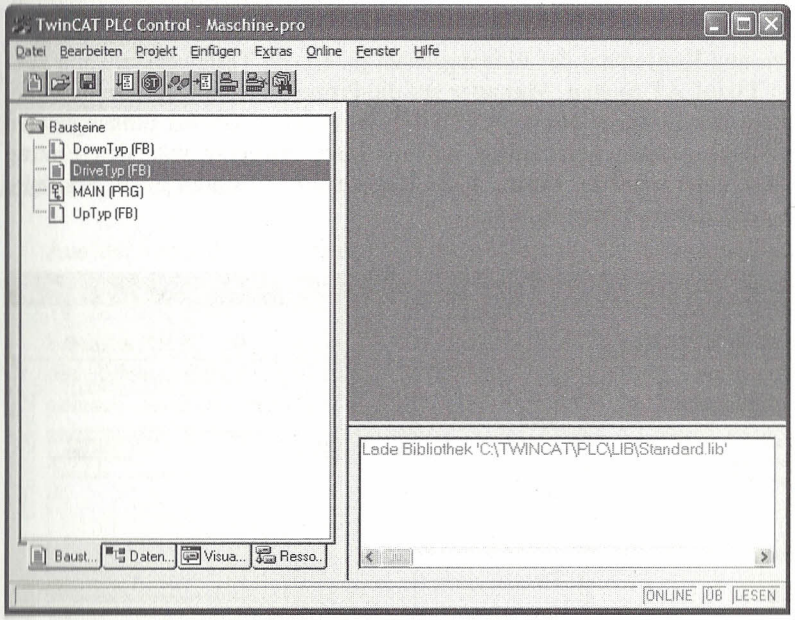


Bild 4.29: *TwinCAT PLC Control*

Um ein neues Steuerungsprogramm zu schreiben, erzeuge man ein neues Projekt. Dazu wird **Datei->Neu** gewählt. Zunächst ist die **Zielplattform** auszuwählen. Das Steuerungsprogramm soll auf dem PC ausgeführt werden:

[16] SPS: Speicherprogrammierbare Steuerung

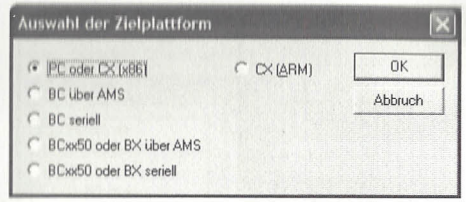


Bild 4.30: *Zielplattform auswählen*

Ein **Projekt** enthält alle Objekte eines Steuerungsprogramms. Diese Objekte können ***Bausteine***, ***Datentypen***, ***Ressourcen*** oder ***Bibliotheken*** sein.

Ein **Baustein** kann eine ***Funktion***, ein ***Programm*** oder ein ***Funktionsblock*** sein. Er besteht aus einem Deklarationsteil und einem Rumpf. Der ***Rumpf*** wird in einer der IEC-Programmiersprachen AWL[17], ST[18], AS[19], KOP[20], FUP[21], CFC[22] geschrieben. Die IEC-Standardbausteine sind in der Bibliothek **standard.lib** enthalten.

Durch rechten Mausklick auf **Bausteine** und Wahl von **Objekt einfügen...** wird ein neuer Baustein erzeugt. Als ersten Baustein erzeuge man ein **Programm** und nenne dieses PLC_PRG. Dazu wird die an PASCAL angelehnte Sprache **ST** gewählt.

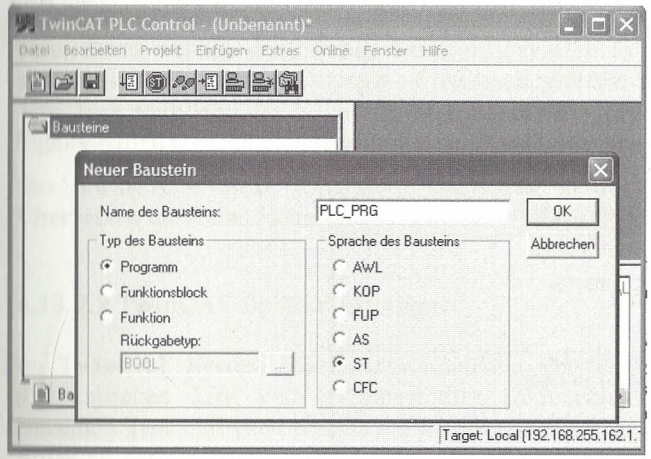


Bild 4.31: *Neuer Baustein*

Nun werden globale Variablen angelegt, die die Ein- und Ausgangsdaten repräsentieren. Dazu wähle man die Registerkarte **Ressourcen**, erweitere den Eintrag **Globale Variablen** und doppelklicke den Eintrag **Globale_Variablen**.

[17] AWL: Anweisungsliste
[18] ST: Strukturierter Text
[19] AS: Ablaufsprache
[20] KOP: Kontaktplan
[21] FUP: Funktionsplan
[22] CFC: Continuous Function Chart

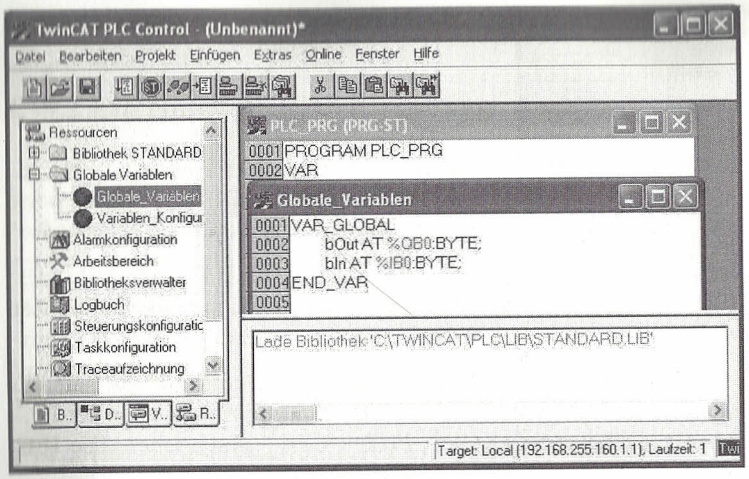


Bild 4.32: *Globale Variablen anlegen*

Nun öffnet sich ein Fenster (Bild 4.32), in dem zwischen VAR_GLOBAL und END_VAR die globalen Variablen deklariert werden. Die Ausgangsvariable `bOut` und die Eingangsvariable `bIn` werden jeweils definiert bei Offset 0 des Prozessabbilds (siehe Bild 4.32).

Der Rumpf von PLC_PRG wird wie folgt definiert:

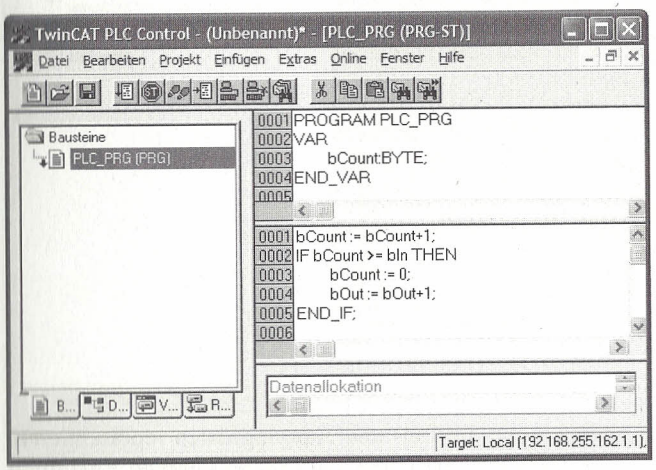


Bild 4.33: *PLC_PRG*

Die für PLC_PRG lokale Variable `bCount` wird bei jedem Aufruf von PLC_PRG inkrementiert, bis der Wert von `bIn` erreicht ist, und danach auf 0 zurückgesetzt. Ist `bIn` erreicht, wird `bOut` um 1 erhöht.

Nun ist dafür zu sorgen, dass PLC_PRG zyklisch (alle 4 ms) ausgeführt wird. Dies wird durch die ***Taskkonfiguration*** erreicht:

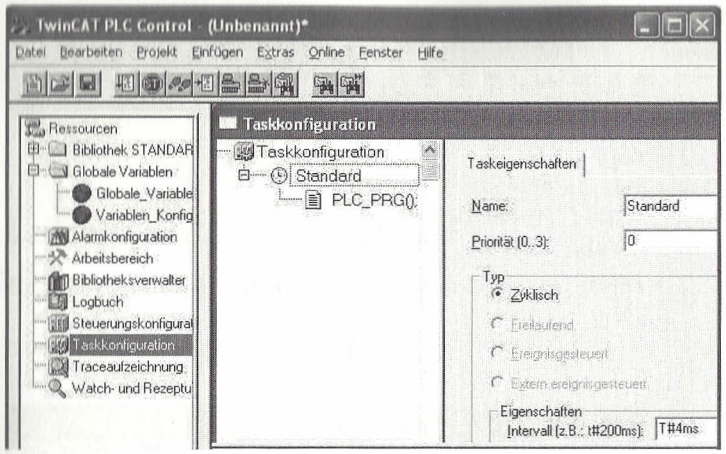


Bild 4.34: *Taskkonfiguration*

Ein ***Task*** ist eine zeitliche Ablaufeinheit eines IEC-Programms. Jedem Task kann eine Folge von Programmen zugeordnet sein, die beim Ausführen des Tasks abgearbeitet werden soll. Im konkreten Fall wird nur ein Task benötigt, dessen Bezeichnung **Standard** übernommen wird. Es muss nun sichergestellt werden, dass dieser Task zyklisch aktiviert wird. Dazu werden der Eintrag **Taskkonfiguration** in der Registerkarte **Ressourcen** aktiviert und der Eintrag **Einfügen->Programmaufruf einfügen** gewählt. Nun wird unter dem Task **Standard** der Name des Programms (PLC_PRG) eingetragen. Bei den Task-Eigenschaften wird im Feld **Intervall** der Wert `T#4ms` an (Bild 4.34) eingegeben.

Das TwinCAT-Projekt wird unter **Bsp1.pro** gespeichert und mit **Projekt->Alles Übersetzen** übersetzt. Beim Übersetzen sollten keine Fehler gemeldet werden.

4.19.2.2 TwinCAT System Manager

Im **TwinCAT System Manager** werden die Ein- und Ausgänge der Tasks und die physikalischen Ein- und Ausgänge des Kommunikationssystems konfiguriert. Die einzelnen Tasks arbeiten jeweils auf einem eigenen, privaten Prozessabbild, so dass die Adressen der im Prozessabbild abgelegten Symbole (vgl. Bild 4.32) nur innerhalb des betreffenden Tasks gültig sind.

Im **TwinCAT System Manager** werden nun die E/A-Informationen der einzelnen Tasks mit den Ein- und Ausgangsdaten des Kommunikationssystems verknüpft, was eine Zuordnung zwischen logischen und physikalischen Ein- und Ausgängen herstellt. Dazu wird die oben erstellte Datei **Bsp1.tsm** bearbeitet.

Zunächst wird das SPS-Projekt **Bsp1** eingetragen. Dazu werden im Kontextmenü von **SPS-Konfiguration** der Menüpunkt **SPS-Projekt anfügen...** und anschließend die Datei **Bsp1.tpy** gewählt. Diese beim Übersetzen des Projekts entstandene Datei enthält alle Variablen und Datenstrukturen des Projekts. Im **TwinCAT System Manager** werden nun der Task **Standard** und dessen Ein- und Ausgänge dargestellt:

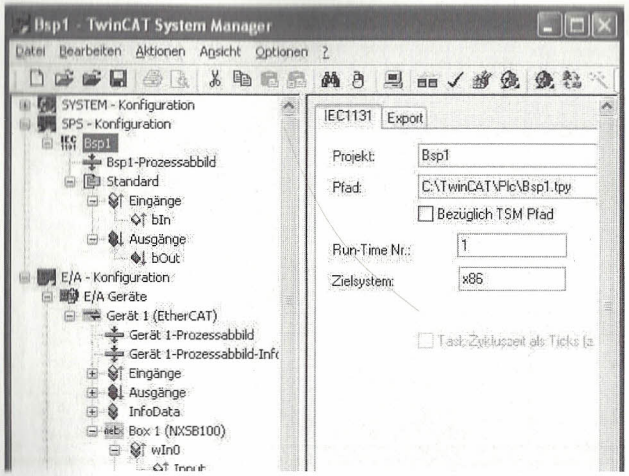


Bild 4.35: *SPS-Programm im System Manager*

Nun sollen die Ein- und Ausgänge des Tasks mit den Ein- und Ausgangsdaten des EtherCAT-Slaves verknüpft werden. Dazu wird der Eintrag **Box 1 (NXSB100)** erweitert und unter **wIn0** auf **Input** geklickt.

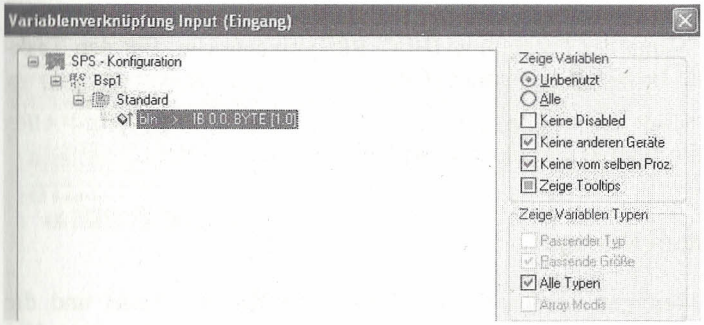


Bild 4.36: *Variablenverknüpfung*

Im rechten Fenster klicke man in der Registerkarte **Variable** auf die Schaltfläche **Verknüpft mit...** Nun erscheint ein Dialog (Bild 4.36), dessen Hauptfenster zunächst leer ist, da standardmäßig nur die Variablen angeboten werden, deren Typ zum Eingang passt. In diesem Fall des Eingangs handelt es sich laut Gerätebeschreibung um 16 Bits; die Variable `bIn` belegt jedoch nur ein Byte. Deshalb wähle man auf der rechten Seite **Alle Typen**. Nun müsste die Variable `bIn` unter dem Task **Standard** von Projekt **Bsp1** erscheinen (Bild 4.36). Diese Wahl wird übernommen. Es folgt ein Dialog, der anzeigt, wie die verknüpfte Variable innerhalb der Eingangsdaten liegt:

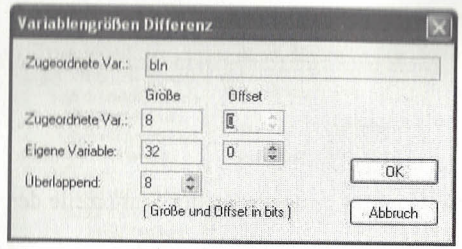


Bild 4.37: *Variablen unterschiedlicher Größe*

Nun wird die Verknüpfung angezeigt:

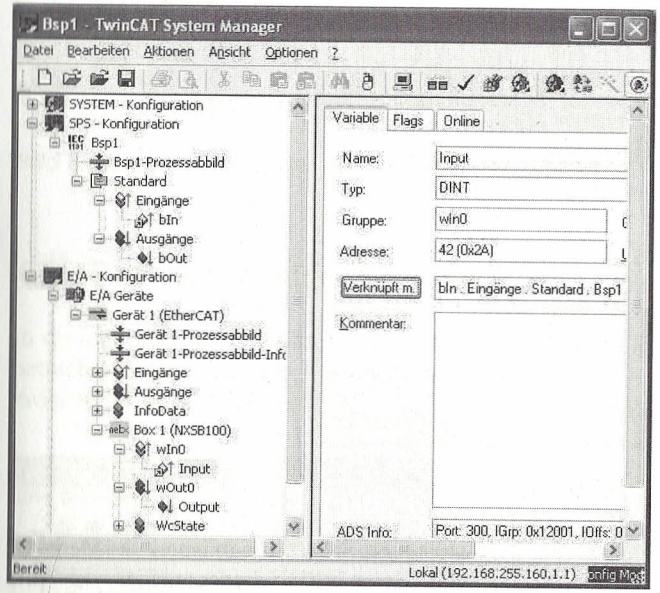
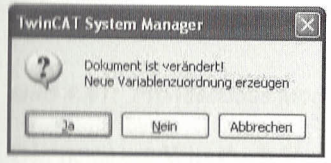
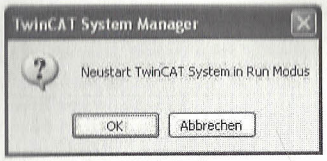


Bild 4.38: *Verknüpfung*

Analog verfahre man mit dem Eintrag **Output** unter **wOut0**. Man aktiviere nun die neue Konfiguration mit **Aktionen->Aktiviert Konfiguration...** oder **Ctrl+Shift+F4**. Die folgende Abfrage



quittiere man mit **Ja** und die anschließende Meldung



mit **OK**. Damit wird die **TwinCAT Echtzeit Erweiterung** gestartet. In der Statuszeile des **TwinCAT System Manager** wird **Running** angezeigt.

4.19.2.3 Laden und Starten des Steuerungsprogramms

Nun kann in **TwinCAT PLC Control** das Programm durch **Online->Einloggen** in die Steuerung geladen und mit **Online->Start** gestartet werden.

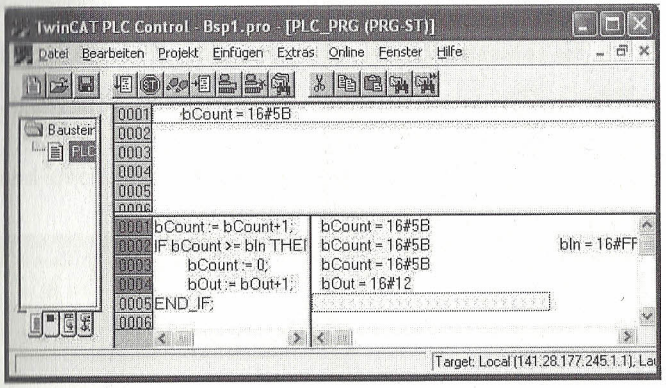


Bild 4.39: *Laufendes Steuerungsprogramm*

An den LEDs kann das Hochzählen der Variablen `bOut` beobachtet werden. Ist `bIn` = 255, so wird am langsamsten hochgezählt. Beim NXHX bedeutet dies, dass alle DIP-Schalter in der Stellung `ON` sind. Klickt man im **TwinCAT System Manager** unter **SPS-Konfiguration** auf die Variable `bOut`, so wird in der Registerkarte **Online** deren Veränderung visualisiert. In Bild 4.40 ist die Veränderung von `bOut` für den netSTICK-Slave zu sehen, bei dem sich der Zählvorgang alle 10 Sekunden verlangsamt.

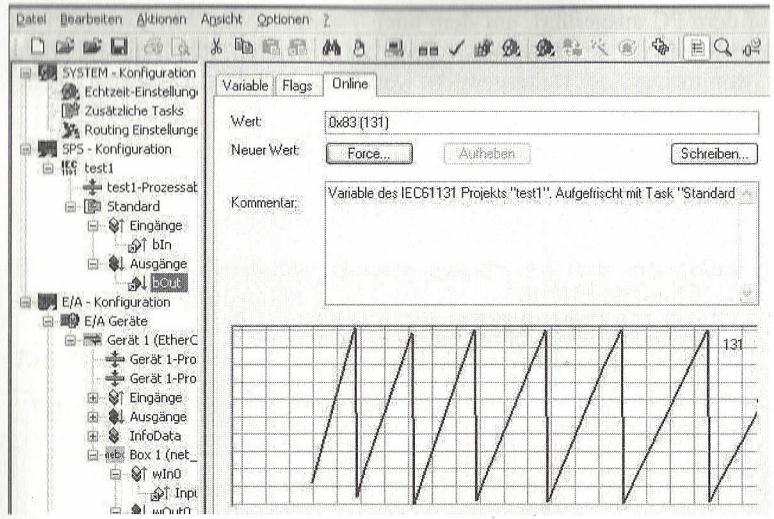


Bild 4.40: *Veränderung der Variablen bOut (netSTICK)*

4.20 Protokollelemente

In diesem Abschnitt werden die zwischen Master und Slave ausgetauschten Telegramme betrachtet. Die Beispiele stammen aus Wireshark-Aufzeichnungen des Datenverkehrs zwischen dem EtherCAT-Master von TwinCAT und dem NXHX als EtherCAT-Slave.

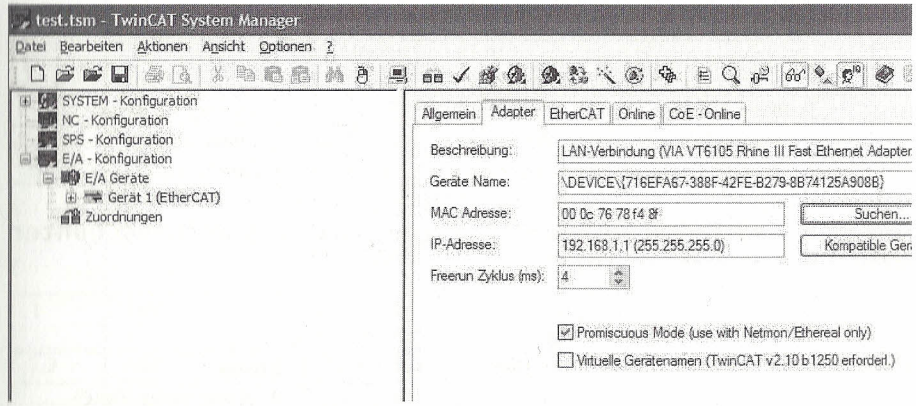


Bild 4.41: *Promiscuous Mode für Ethernet Adapter*

Die neuesten Wireshark-Versionen sind mit einem EtherCAT-Dissector ausgestattet, der eine bequeme Interpretation der EtherCAT-Telegramme ermöglicht. In diesem Beispiel wird die „neutrale" Sicht verwendet, d. h. es werden die Ethernet-Frames aufgenommen und vom Betrachter interpretiert.

Wireshark wurde auf dem PC ausgeführt, auf dem auch TwinCAT läuft. In TwinCAT ist dazu der ***Promiscuous Mode*** für den Ethernet-Adapter zu aktivieren, damit die Ethernet-Frames auch an das Monitoring-Tool weitergereicht werden (Bild 4.41). Ansonsten „sieht" das Monitoring-Tool keine Frames.

4.20.1 EtherCAT-Frame

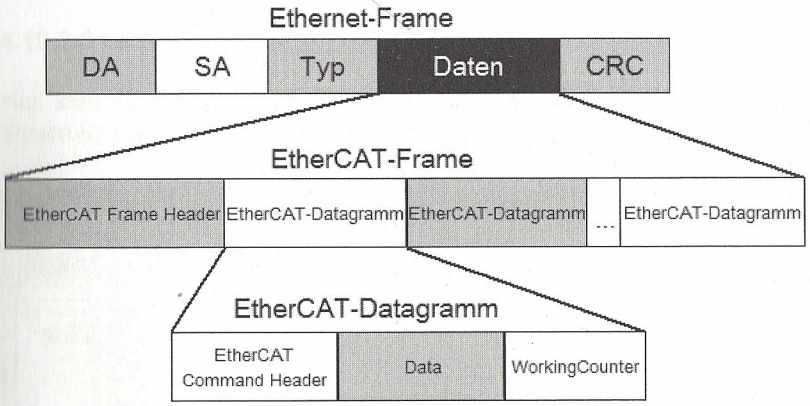


Bild 4.42: *EtherCAT-Frame, EtherCAT-Datagramm*

Bild 4.42 zeigt den Aufbau des EtherCAT-Frames. Dieser belegt den Datenteil des Ethernet-Frames. Der EtherCAT-Frame besteht aus dem **EtherCAT Frame Header** und einem oder mehreren **EtherCAT-Datagrammen**:

Tabelle 4.17: *EtherCAT-Frame*

2 Bytes	$10 + n_1 + 2$ Bytes	$10 + n_2 + 2$ Bytes	...	$10 + n_m + 2$ Bytes
EtherCAT Frame Header	EtherCAT-Datagramm	EtherCAT-Datagramm	...	EtherCAT-Datagramm

Jedes ***EtherCAT-Datagramm*** setzt sich aus Header, Daten und `WorkingCounter` zusammen:

Tabelle 4.18: *EtherCAT-Datagramm*

10 Bytes	n Bytes	2 Bytes
EtherCAT Command Header	Data	WorkingCounter (WKC)

Der EtherCAT Frame Header enthält als wichtigste Angabe die Gesamtlänge des EtherCAT-Frames:

Tabelle 4.19: *EtherCAT Frame Header*

Bits	Feld	Beschreibung
0-10	`Length`	Länge des EtherCAT-Frames (ohne CRC) in Bytes
11	`Reserved`	Reserviert
12-15	`ProtocolType`	Wert `0x01`: EtherCAT-Datagramm. Nur `ProtocolType = 0x01` wird von einem EtherCAT Slave Controller bearbeitet

Der ***EtherCAT Command Header*** enthält u. a. das auszuführende Kommando und Adressierungsinformation:

Tabelle 4.20: *EtherCAT Command Header*

Bytes	Bezeichnung	Bedeutung	
0	CMD	Kommando	
		`APRD (0x01)`	Lesen eines physikalische Bereichs, Auto-Inkrement-Adressierung
		`APWR (0x02)`	Schreiben eines physikalischen Bereichs, Auto-Inkrement-Adressierung
		`APRW (0x03)`	Lesen und Schreiben eines physikalischen Bereichs, Auto-Inkrement-Adressierung
		`FPRD (0x04)`	Knoten-adressiertes (Fixed Address) Lesen eines physikal. Bereichs
		`FPWR (0x05)`	Knoten-adressiertes (Fixed Address) Schreiben eines physikal. Bereichs
		`FPRW (0x06)`	Lesen und Schreiben eines physikalischen Bereichs, Konten-Adressierung
		`BRD (0x07)`	Broadcast Read: Oder-verknüpftes Lesen eines physikal. Bereichs bei allen Slaves
		`BWR (0x08)`	Broadcast Write:. Oder-verknüpftes Lesen und Schreiben eines physikal. Bereichs bei allen Slaves
		`BRW (0x09)`	Broadcast Read/Write: Lesen und Schreiben eines physikalischen Speicherbereichs
		`LRD (0x0A)`	Lesen eines logischen Speicherbereichs
		`LWR (0x0B)`	Schreiben eines logischen Speicherbereichs
		`LRW (0x0C)`	Lesen und Schreiben eines logischen Speicherbereichs
		`ARMW (0x0D)`	Auto-increment Read Multiple Write eines physikalischen Speicherbereichs
		`FRMW (0x0E)`	Knoten-adressiertes Read Multiple Write eines physikalischen Speicherbereichs
1	IDX	Index: Wird vom Slave unverändert weitergesendet. Damit kann der Master das Telegramm beim Empfang zuordnen.	
2,3	ADP	Address Page: Adressiert den Slave.	Bei logischer Adressierung (`LRD`, `LWR`, `LRW`) bilden `ADP` und `ADO` zusammen die logische Adresse (32 Bit).
4,5	ADO	Address Offset: Physikalische Adresse im Slave Adressraum	
6,7	LEN	Bits: 0-10 — Länge des Datenfelds Bits: 11-14 — Reserviert (0) Bits: 15 — Ist dieses Bit gesetzt, so folgen weitere EtherCAT-Datagramme. Ansonsten handelt es sich um das letzte	

8,9	INT	Interrupt-Feld: Hier kann ein Slave Ereignisse ohne Zeitverzögerung melden. Der Master kann per Multiple-Adressierung von allen Slaves die zum Ereignis gehörigen Diagnosedaten lesen.

Die Übertragung numerischer Datentypen erfolgt bei EtherCAT in der Byte Order Little Endian. Erst wird das niederwertigste Byte (Bits 0 - 7) übertragen, dann das Byte mit den Bits 8 - 15 usw.

Als Beispiel betrachten wir den in Bild 4.43 dargestellten Ethernet-Frame, der einen EtherCAT-Frame enthält. Die Darstellung beginnt mit der MAC-DA.

```
01 01 05 01 00 00 00 0c 76 78 f4 8f 88 a4 34 10
01 ff 00 00 02 05 0e 80 00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00 00 00 01 ff ff ff 02 05
0e 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00 00 00 00
```

Bild 4.43: *Ethernet-Frame mit EtherCAT-Frame (Kommandos an zwei Slaves)*

Nach dem Ethernet-Typ (88 a4) folgt der EtherCAT Frame Header (34 10). Der EtherCAT-Frame hat eine Länge von 0x34 (52), und der ProtocolType ist ein EtherCAT-Datagramm (0x10, d. h. die Bits 12-15 sind 0001). Der EtherCAT Command Header des ersten EtherCAT-Datagramms beginnt in der zweiten Zeile mit CMD = 0x01 (APRD, siehe Tabelle 4.20). Das Feld IDX ist mit 0xff belegt. Weiter sind ADP = 0x0000 und ADO = 0x0502, d. h. es soll ab Adresse 0x0502 gelesen werden. Das Feld LEN ist mit 0e 80 belegt. Damit ist die Länge des Datenfeldes 14 (0x00e), und das gesetzte Bit 15 (0x80) zeigt an, dass ein weiteres EtherCAT-Datagramm folgt. INT enthält 00 00. Es folgen 14 Bytes Datenfeld (alles 00) und WKC = 0x0000. Das nächste EtherCAT-Datagramm beginnt mit dem Command Header 01 ff ff ff 02 05 0e 00 00 00 und unterscheidet sich nur an zwei Stellen vom vorigen: ADP ist mit ff ff belegt, und LEN enthält 0e 00 Aus der Längenangabe geht hervor, dass kein weiteres EtherCAT-Datagramm folgt.

Bei beiden Kommandos wird die Positionsadressierung mit Auto-Inkrement (APRD) verwendet. Im ersten Fall ist das Feld ADP mit 0x0000 belegt, also wird der erste Slave im Ring angesprochen. Im zweiten Fall hat ADP den Inhalt 0xffff (-1). Der erste Slave inkrementiert und leitet damit den Wert 0 an den zweiten Slave weiter. So ist der zweite Slave Empfänger des Kommandos. Bild 4.44 zeigt den EtherCAT-Frame mit den Antworten der beiden Slaves:

```
01 01 05 01 00 00 02 0c 76 78 f4 8f 88 a4 34 10
01 ff 02 00 02 05 0e 80 04 00 c0 00 0c 00 00 00
01 00 03 00 00 00 00 00 01 00 01 ff 01 00 02 05
0e 00 04 00 c0 00 0c 00 00 00 01 00 03 00 00 00
00 00 01 00
```

Bild 4.44: *Ethernet-Frame mit EtherCAT-Frame (Slave-Antworten)*

Der WKC hat jeweils den Wert 1 (01 00), und der Datenteil enthält jeweils den Inhalt des Speicherbereichs 0x502-0x50f der beiden Slaves. Man beachte, dass die Adressen (Feld ADP) jeweils zweimal (d. h. von jedem Slave) inkrementiert wurden und nun 02 00 bzw. 01 00 enthalten.

4.20.2 Slave-Initialisierung

4.20.2.1 Slave-Identifikation

Wählt man im **TwinCAT System Manager** im Kontextmenü von Gerät 1 (EtherCAT) **Boxen Scannen...**, so sendet der Master folgenden Ethernet-Frame:

```
ff ff ff ff ff ff 00 0c 76 78 f4 8f 88 a4 10 10
01 ff 00 00 00 00 04 00 00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00 00 00 00 00 00
```

Bild 4.45: *Ethernet-Frame mit EtherCAT-Frame (Boxen scannen...)*

Die ersten 12 Bytes sind durch die MAC-Adressen belegt (DA und SA). Es folgt das Typ-Feld (0x88a4 - EtherCAT). Die übrigen Bestandteile des Ethernet-Frames werden in Tabelle 4.21 interpretiert:

Tabelle 4.21: *Interpretation zu Bild 4.45*

10	Länge des EtherCAT-Frames: 0x10 (16)
10	ProtocolType = 0x01 (EtherCAT-Datagramm)
01	EtherCAT Command Header: CMD = 0x01 (APRD)
ff	EtherCAT Command Header: IDX (Index)
00 00 00 00	EtherCAT Command Header: ADP = 0x0000, ADO = 0x0000
04 00	EtherCAT Command Header: Length = 0x0004
00 00	INT
00 00 00 00	Data
00 00	WKC

Bei der zweiten und dritten Zeile des abgebildeten Frames handelt es sich um Padding zur Erreichung der Mindestlänge von 64 Bytes.

Hinweis: *Die MAC-Adressen spielen bei der EtherCAT-Kommunikation keine Rolle. Insbesondere sind MAC-DA und MAC-SA bei der Slave-Antwort nicht vertauscht. Um trotzdem anzuzeigen, dass es sich bei einem Frame um eine Slave-Antwort handelt, wird das erste Adressbyte der MAC Source verändert. Im obigen Beispiel (Bild 4.44) wird 00 zu 02.*

Die mit APRD gewählte Positions-Adressierung (Auto-Inkrement) verwendet als Adresse (ADP) eine 0 und richtet sich somit an den ersten Slave. Bei diesem sollen die ersten vier Bytes seines Adressraums ausgelesen werden. Nach Tabelle 4.4 liegen dort die Informationen Type, Revision und Build.

Der erste Slave antwortet und inkrementiert den WKC, um anzuzeigen, dass der Lese-Befehl erfolgreich ausgeführt wurde. Im Folgenden wird nur noch der EtherCAT-Frame dargestellt. Dieser beginnt mit dem **EtherCAT Frame Header**, bestehend aus Längenangabe des EtherCAT-Frames (hier: 0x10) und dem ProtocolType (EtherCAT-Datagramm, 0x01. **Achtung**: Der ProtocolType belegt die Bits 12-15 im EtherCAT Frame Header). Padding-Bytes werden ebenfalls nicht mehr dargestellt.

```
10 10 01 ff 01 00 00 00 04 00 00 00 80 01 03 00
01 00
```

Bild 4.46: *EtherCAT-Frame mit Slave-Antwort*

Feld ADP (01 00) enthält die inkrementierte Slave-Adresse. Nach dem Längenfeld 04 00 folgen das Interrupt Feld 00 00, die Daten 80 01 03 00 und der WKC 01 00, der einen erfolgreichen Lesezugriff anzeigt. Der EtherCAT-Slave hat also Type = 0x80, Revision = 0x01, Build = 0x0003 (siehe Tabelle 4.4). Insbesondere zeigt der erfolgreiche Lesezugriff, dass mindestens ein Slave vorhanden ist.

Nun liest der Master weitere Informationen aus dem Slave-Adressraum (vgl. Tabelle 4.4). Zunächst werden EEPROM-Informationen ausgelesen (vgl. Tabelle 4.5).

```
0d 10 08 ff 00 00 00 05 01 00 00 00 00 00 00
```

Bild 4.47: *Übernahme des EEPROM-Zugriffs*

Dieser EtherCAT-Frame hat die Länge 0x0d (13). Der Wert 0x08 im CMD-Feld zeigt ein BWR-Kommado an, also ein Schreiben eines physikalischen Bereichs bei allen Slaves. Dabei ist das ADP-Feld immer Null, und das ADO-Feld (hier: 00 05) gibt die physikalische Adresse 0x0500 an. Es wird ein Byte (Length 01 00) mit Wert 00 geschrieben. Damit übernimmt der EtherCAT-Master den Zugriff auf das EEPROM (SII, siehe 4.10). Der Slave quittiert positiv (WKC = 1):

```
0d 10 08 ff 00 00 00 05 01 00 00 00 01 00
```

Bild 4.48: *Positive Quittung: WKC = 1*

Nun schreibt der Master beim Slave mit APWR (Auto Increment Physical Write) ab Adresse 0x502 die Daten 00 01 08 00 00 00, womit ein **EEPROM-Lesezugriff** von der EEPROM-Adresse 0x0008 gestartet wird. Die EEPROM-Adresse wird als **WORD-Adresse** interpretiert, d. h. es wird im EEPROM von Byte-Adresse 16 (0x10) gelesen.

```
12 10 02 ff 00 00 02 05 06 00 00 00 00 01 08 00
00 00 00 00
```

Bild 4.49: *Anforderung EEPROM-Daten*

Dies wird vom Slave wieder positiv quittiert. Die gelesenen EEPROM-Daten werden im Slave-Adressbereich 0x0508–0x050F (EEPROM Data) abgelegt. Bild 4.50 zeigt, wie der Master die gelesenen SII-Daten anfordert, indem er mit APRD den Adressbereich 0x0502–0x50F liest.

```
1a 10 01 ff 00 00 02 05 0e 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00 00 00 00 00
```

Bild 4.50: *Anfordern der gelesenen EEPROM-Daten*

Die Antwort enthält die EEPROM-Daten 44 00 00 00 6e 65 74 78, die aus VendorID (44 00 00 00, 0x00000044) und ProductCode (6e 65 74 78, ASCII „*netx*“) bestehen (vgl. Tabelle 4.5):


```
1a 10 01 ff 01 00 02 05 0e 00 00 00 c0 00 08 00
00 00 44 00 00 00 6e 65 74 78 01 00
```

Bild 4.51: *Slave-Antwort mit EEPROM-Informationen*

Mit diesen Informationen kann der Master die Gerätebeschreibungsdatei zum gefundenen Slave identifizieren (siehe Tabelle 4.15).

Analog liest der Master den gesamten EEPROM-Inhalt. Die gelesenen EEPROM-Daten lassen sich in TwinCAT anzeigen, wenn man den Slave selektiert und in der Registerkarte **EtherCAT** die Schaltfläche **Erweiterte Einstellungen...** klickt. Dort kann man unter **ESC-Zugriff->E²PROM** die Daten des EEPROM einsehen. Im **Smart View** werden einzelne Felder interpretiert, unter **Hex Editor** erhält man eine neutrale Ansicht:

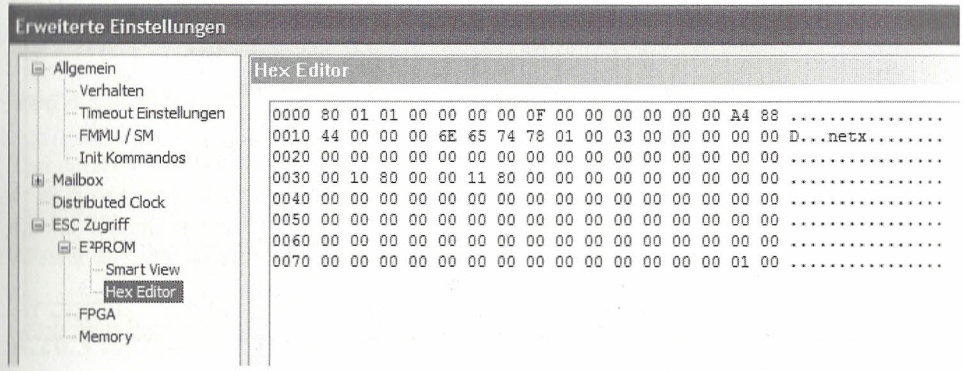


Bild 4.52: *EEPROM-Daten des Slave*

Nun wird das Register 0x0111 (DL Status) gelesen. Dieses zeigt an, ob auf den Ethernet-Ports ein Trägersignal anliegt und ob die betreffenden RX-und TX-Schnittstellen kurzgeschlossen sind (***Loop***). Schließlich werden diverse Speicherbereiche beim Slave mit dem Wert 0 vorbelegt.

4.20.2.2 ESM

Nach diesen Initialisierungen folgt eine weitere Initialisierungssequenz, bei der bereits Informationen aus der Gerätebeschreibung eingehen. Der Master veranlasst, dass der Slave Zustände der ESM durchläuft. Wie dies genau geschieht und welche Initialisierungen bei Zustandsübergängen erfolgen, kann in TwinCAT dargestellt werden. Klickt man in der Registerkarte **EtherCAT** des Slaves die Schaltfläche **Erweiterte Einstellungen...**, so erhält man unter **Allgemein->Init Kommandos** ein Dialogfenster (Bild 4.53), welches eine Tabelle mit Kommandos zeigt, die vom Master abgesetzt werden, wenn von einem ESM-Zustand in einen anderen gewechselt wird. In die Tabelle gehen Informationen aus der Slave-Gerätebeschreibung ein. In der Kopfzeile sind in den Spalten vor CMD die möglichen Zustandsübergänge der ESM zu sehen (vgl. Tabelle 4.10). Ein Kreuz in der betreffenden Spalte bedeutet, dass das Kommando beim entsprechenden Zustandsübergang ausgeführt wird. Bei einem normalen Hochlauf wird die Sequenz

INIT → PRE-OPERATIONAL → SAFE-OPERATIONAL → OPERATIONAL, also IP → PS → SO durchlaufen.

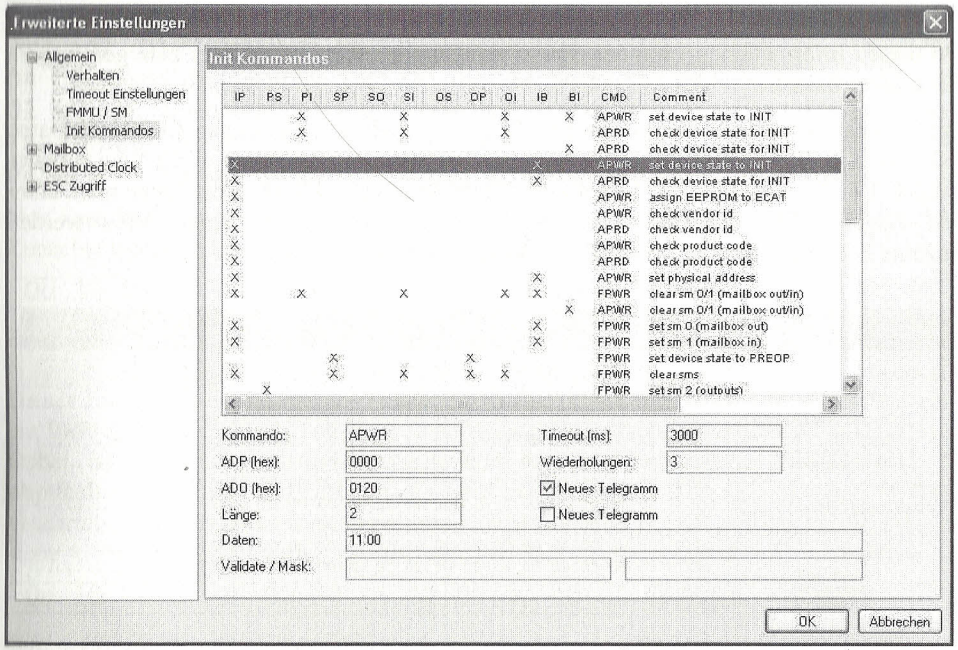


Bild 4.53: *Init-Kommandos für EtherCAT-Slave*

Über die Registerkarte **Online** können Slave-Zustände auch direkt gesetzt werden. So kann insbesondere das Verhalten der Status-LEDs beobachtet werden.

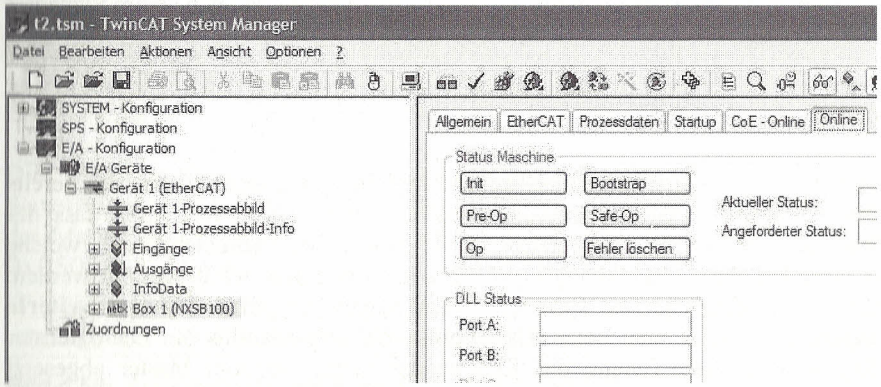


Bild 4.54: *Slave-Zustände setzen*

Bild 4.55 zeigt die FMMUs und SMs für einen Slave, der gemäß Spalte „Beispiel" von Tabelle 4.15 konfiguriert ist.

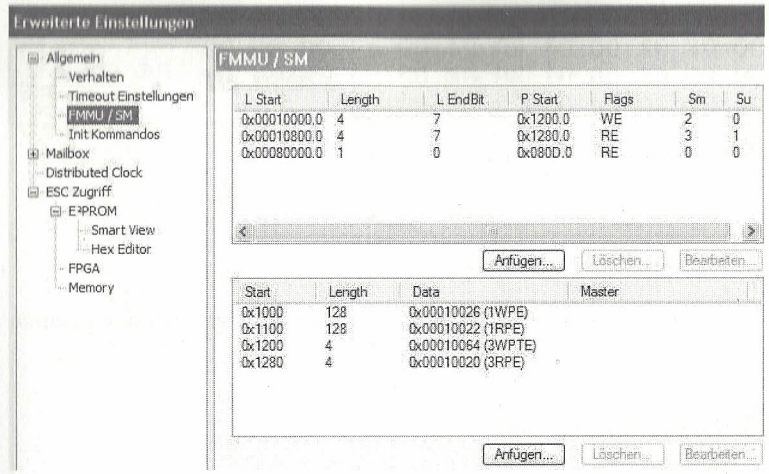


Bild 4.55: *FMMU/SM*

Der letzte FMMU-Eintrag (Physikalische Startadresse 0x080D, Bit 0) bezieht sich auf Byte 5 der Parameter für SM-Kanal 1 (Mailbox Send), welches das Bit QueuedState (siehe Tabelle 4.11) enthält. Durch Lesen der logischen Adresse 0x0008000 kann der Master also ermitteln, ob der Slave in die Mailbox geschrieben hat.

4.20.3 Zyklischer Datenverkehr

Nach der Initialisierungsphase stehen alle für die zyklische Datenübertragung erforderlichen Informationen zur Verfügung. Bild 4.56 zeigt einen Request des Masters im Rahmen des zyklischen Datenaustauschs.

```
3b 10 0b 00 00 00 01 00 04 80 00 00 00 00 00 00
00 00 0a 00 00 08 01 00 04 80 00 00 00 00 00 00
00 00 0a 00 00 00 08 00 01 80 00 00 00 00 00 07
00 00 00 30 01 02 00 00 00 00 00 00 00 00
```

Bild 4.56: *Master-Request (zyklischer Datenverkehr)*

Dieser Frame enthält vier Kommandos, nämlich ein LWR (0b), zwei LRD (0a) sowie ein BRD (07). Der LWR bezieht sich auf die logische Adresse 0x00010000 (Ausgangsdaten schreiben). Es wird ein Wert der Länge 4 geschrieben, nämlich 00 00 00 00. Der erste LRD liest von der logischen Adresse 0x00010800 (Eingangsdaten), der zweite von 0x00080000 (Mailbox-Zustand). Schließlich wird mit BRD von der physikalischen Adresse 0x130 (ADP = 0, ADO = 0x130) gelesen, also der ESM-Zustand angefordert.

Eine typische Slave-Antwort ist in Bild 4.57 zu sehen. Die WKC-Felder für jedes Kommando sind inkrementiert und zeigen somit die erfolgreiche Ausführung an. Die Ausgangsdaten wurden übernommen, und die angeforderten Eingangsdaten (0x0000 00e7), der Mailbox-Zustand (0x00) und der ESM-Zustand (0x0008, OPERATIONAL) sind im Frame hinterlegt.

```
3b 10 0b 00 00 00 01 00 04 80 04 00 00 00 00 00
01 00 0a 00 00 08 01 00 04 80 04 00 e7 00 00 00
01 00 0a 00 00 00 08 00 01 80 04 00 00 01 00 07
00 00 00 30 01 02 00 04 00 08 00 01 00
```

Bild 4.57: Slave-Antwort (zyklischer Datenverkehr)

4.20.4 Mailbox-Kommunikation, CoE

Wählt man erstmals für einen Slave die Registerkarte **CoE-Online**, so wird das gesamte Objektverzeichnis angezeigt (Bild 4.59).

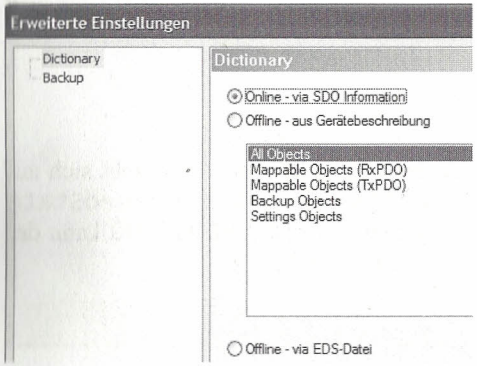


Bild 4.58: OD-Information Online/Offline

Dabei kann man über die Schaltfläche **Erweitert...** auswählen, ob die Informationen über das Objektverzeichnis aus einer EDS-Datei oder vom Gerät gelesen werden (Bild 4.58). In diesem Beispiel wird die Online-Variante verwendet, d. h. sämtliche Informationen werden mit Hilfe der sog. ***SDO Information Services*** aus dem Gerät gelesen.

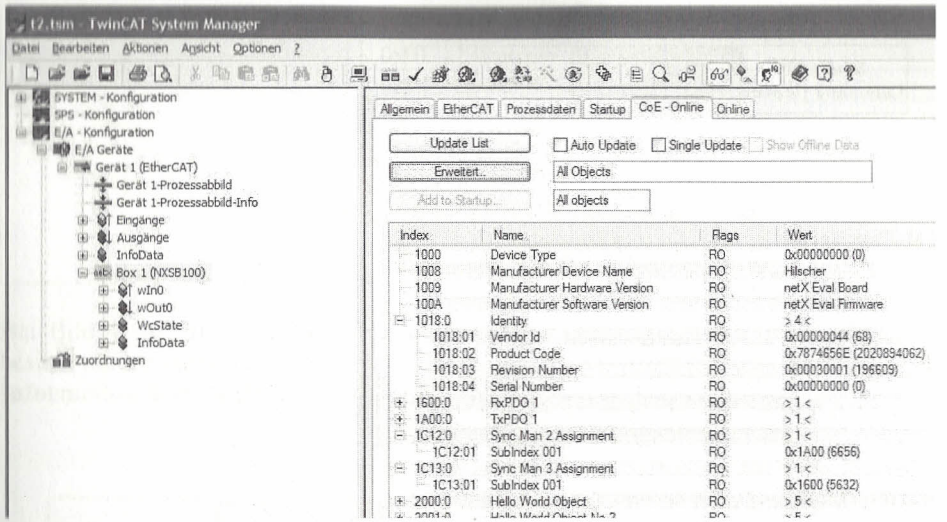


Bild 4.59: CoE Objektverzeichnis

4.20.4.1 SDO Information Services

Mit den **SDO Information Services** wird ermittelt, welche Objekte welchen Typs im Gerät vorhanden sind. Der entsprechende Ablauf und die verwendeten Protokollelemente werden nun an einem Beispiel beschrieben. Bild 4.60 zeigt den ersten Mailbox-Zugriff durch den Master.

```
27 10 05 91 e9 03 00 10 0e 80 00 00 08 00 00 00
00 03 00 80 01 00 00 00 00 00 00 00 05 00 e9 03
7f 10 01 00 00 00 00 00 00
```

Bild 4.60: EtherCAT-Frame mit Mailbox-Zugriff

Dieser Frame enthält zwei `FPWR`-Kommandos (Knoten-adressiertes Schreiben eines physikalischen Bereichs). Es wird nur das erste Kommando betrachtet, welches sich an den Slave mit Adresse `0x03e9` (1001) wendet und 14 (`0x000e`) Bytes ab Adresse `0x1000` (**Mailbox Out**) schreibt. Die Daten, die in die Mailbox geschrieben werden, sind grau unterlegt. Sie sind in einen ***Mailbox Header*** und in sog. ***Mailbox Service Data*** unterteilt. Da über die Mailbox verschiedene Protokolle transportiert werden können, muss aus dem Mailbox Header die Bedeutung der Daten hervorgehen.

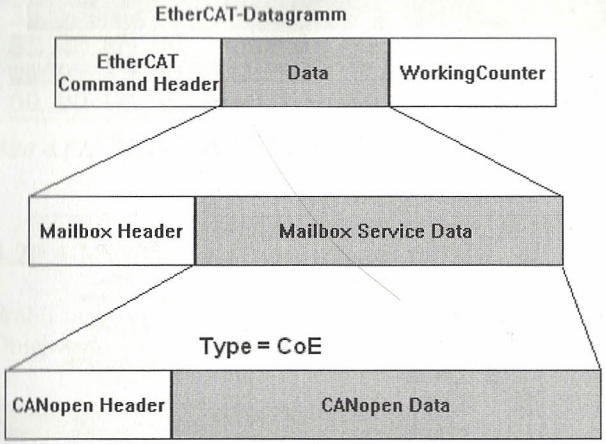


Bild 4.61: *Mailbox-Daten im Fall Type = CoE*

Wie die Mailbox Service Data zu interpretieren sind, ergibt sich aus dem Mailbox Header:

Tabelle 4.22: *Mailbox Header*

Bytes	Parameter	Beschreibung
0-1	`Length`	Länge der Mailbox Service Data
2-3	`Address`	Stationsadresse des Absenders, falls der Master Client ist. Stationsadresse des Empfängers, falls der Slave Client ist.
4	`Channel/Priority`	Die `Channel`-Bits (0-5) sind für künftige Verwendung reserviert. Die Priority-Bits (6-7) erlauben die Angabe von 4 Prioritätsstufen (`0x00` – niedrigste Priorität, ..., `0x03` – höchste Priorität).
5	`Type`	Typ des Mailbox-Protokolls

Wert	Bedeutung
0x01	Herstellerspezifisch
0x02	Ethernet over EtherCAT (EoE)
0x03	CANopen over EtherCAT (CoE)
0x04	File Access over EtherCAT (FoE)
0x05	Servo Profile over EtherCAT (SoE)

Die Daten `08 00 00 00 00 03 00 80 01 00 00 00 00 00` im obigen Frame sind also wie folgt zu lesen: `Length` = 8, `Address` = 0, `Channel/Priority` = 0, `Type` = 3 (CoE). Die Mailbox Service Data `00 80 01 00 00 00 00 00` sind damit entsprechend **CANopen over Ethernet** zu interpretieren. Die ersten beiden Bytes (`00 80`) bilden den ***CANopen Header***:

Tabelle 4.23: *CANopen Header*

Bits	Parameter	Beschreibung
0-8	`Number`	Abhängig vom CANopen-Dienst
9-11	`Reserved`	Reserviert
12-15	`Service`	CANopen-Dienst

Wert	Bedeutung
0x01	Emergency
0x02	SDO Request
0x03	SDO Response
0x04	TxPDO
0x05	RxPDO
0x06	TxPDO Remote Request
0x07	RxPDO Remote Request
0x08	SDO Information

Bei Bild 4.60 handelt es sich also um den **SDO Information Service**. Dieser Dienst besitzt wieder einen Header (4 Bytes, hier: `01 00 00 00`), der den Nutzdaten (**SDO Information Service Data**) vorangestellt ist.

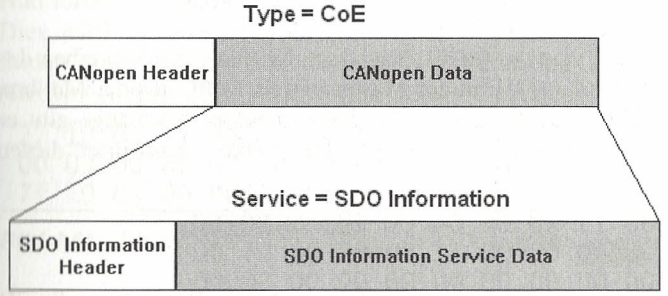


Bild 4.62: *SDO Information*

Daher wird nun der SDO Information Header betrachtet:

Tabelle 4.24: *SDO Information Header*

Bits	Parameter	Beschreibung
0-6	`Opcode`	

Wert	Bedeutung
0x01	GetODList Request
0x02	GetODList Response
0x03	GetObjectDescription Request
0x04	GetObjectDescription Response
0x05	GetEntryDescription Request
0x06	GetEntryDescription Response

7	`Incomplete`	

Wert	Bedeutung
0x00	Letztes SDO Information Fragment

8-15	`Reserved`	Reserviert
16-31	`FragmentsLeft`	Anzahl noch folgender Fragmente

Bei Bild 4.60 geht es also um einen GetODList Request, mit welchem eine Liste aller CANopen-Objekte angefordert wird. Beim GetODList Request bestehen die SDO Information Service Data aus 2 Bytes, in denen der `ListType` spezifiziert wird:

Tabelle 4.25: *SDO Information Service Data bei GetODList Request*

Bits	Parameter	Beschreibung	
0-15	`ListType`	**Wert**	**Bedeutung**
		0x00	Anzahl der Objekte der verschiedenen Listen. Dieser Wert ist in der EtherCAT-Spezifikation [IEC06d] nicht genannt. Auf Rückfrage wurde aber von der ETG diese Verwendung bestätigt.
		0x01	Alle OD-Objekte sollen in der Response übertragen werden.
		0x02	Nur OD-Objekte, die in PDOs gemappt werden können, sollen in der Response übertragen werden.
		0x03	Nur OD-Objekte, die im Falle eines Geräteaustauschs gespeichert werden müssen, sollen in der Response übertragen werden.

Im Beispiel (Bild 4.60) ist `ListType` = 0x00. Nachdem der Slave das Schreiben des Mailbox Requests quittiert hat (`WKC` = 1), liest der Master mittels `FPRD` aus der Mailbox (physikalische Adresse: 0x1100), um die Slave-Antwort abzuholen. Als Länge gibt er dabei die konfigurierte Maximallänge (hier: 128) an. Der Slave beantwortet diesen Request wie folgt:

```
8c 10 04 92 e9 03 00 11 80 00 04 00 12 00 00 00
00 03 00 80 02 00 00 00 00 00 11 00 03 00 05 00
00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00 00 00 00 00 01 00
```

Bild 4.63: *Slave-Antwort mit GetODList Response (ListType = 0)*

Die Nutzdaten der Länge 0x0080 in der `FPRD` Response sind nach dem gleichen Schema wie der vorige Request aufgebaut. Der Mailbox Header (12 00 00 00 00 03) besagt, dass 18 Bytes Daten für CoE folgen. Der CANopen Header (00 80) zeigt an, dass es sich um den SDO Information Service handelt, und laut SDO Information Header (02 00 00 00) wird eine GetODList Response transportiert. Bei dieser enthalten die ersten beiden Bytes der SDO Information Service Data den `ListType`. Danach folgt die `IndexList`, die im Beispiel aus folgenden Elementen besteht: 0x0011, 0x0003, 0x0005, 0x0000, 0x0000. Der erste Wert (0x0011) ist die Anzahl der Objekte für `ListType` = 1 (alle Objekte). Die Längenangabe im Mailbox Header zeigt die Länge der relevanten Mailbox-Daten an. Diese ist erforderlich, da der Master die gesamte Mailbox liest. Am Ende der Daten steht schließlich der `WKC`.

Der Master schickt nun einen SDO Information Request mit einem GetODList Request, der sich auf `ListType` = 1 (alle Objekte) bezieht. Nachdem der Slave den Request quittiert und eine Antwort in der Mailbox bereitgestellt hat, kann der Master wieder die Mailbox auslesen. Die Slave-Antwort sieht wie folgt aus:


```
49 10 04 99 e9 03 00 11 30 80 04 00 2a 00 00 00
00 03 00 80 02 00 00 00 01 00 00 10 08 10 09 10
0a 10 18 10 00 16 00 1a 12 1c 13 1c 00 20 01 20
02 20 03 20 04 20 05 20 10 20 20 20 01 00 04 00
e9 03 7f 11 01 00 04 00 00 01 00
```

Bild 4.64: *Slave-Antwort mit GetODList Response (ListType = 1)*

Hier besteht der Mailbox-Datenteil aus 0x2a (42) Bytes. Es handelt sich wieder um eine `GetODList` Response. Der `ListType` ist 0x0001, und es folgen die Indizes der CANopen-Objekte: 0x1000, 0x1008, 0x1009, 0x100a, 0x1018, 0x1600, 0x1a00, 0x1c12, 0x1c13, 0x2000, 0x2001, 0x2002, 0x2003, 0x2004, 0x2005, 0x2010, 0x2020.

Nun fordert der Master der Reihe nach alle **Objektbeschreibungen** für diese Objekte an. Dies wird exemplarisch für das Objekt 0x1018 beschrieben. Mit folgendem Frame fordert der Master mit dem SDO Information Service und `Opcode` = 0x03 (GetObjectDescription Request) die Objektbeschreibung für das Objekt 0x1018 an:

```
27 10 05 c1 e9 03 00 10 0e 80 00 00 08 00 00 00
00 03 00 80 03 00 00 00 18 10 00 00 05 00 e9 03
7f 10 01 00 00 00 00 00 00
```

Bild 4.65: *EtherCAT-Frame mit GetObjectDescription Request*

Der Slave stellt folgende Antwort bereit, welche vom Master mittels `FPRD` abgeholt wird:

```
33 10 04 c3 e9 03 00 11 1a 80 04 00 14 00 00 00
00 03 00 80 04 00 00 00 18 10 23 00 04 09 49 64
65 6e 74 69 74 79 01 00 04 00 e9 03 7f 11 01 00
04 00 00 01 00
```

Bild 4.66: *EtherCAT-Frame mit GetObjectDescription Response*

Diese enthält eine GetObjectDescription Response (04). Deren Datenteil hat folgenden Aufbau:

Tabelle 4.26: *SDO Information Service Data bei GetObjectDescription Response*

Bytes	Feld	Beschreibung
0-1	`Index`	Index des Objekts
2-3	`DataType`	CANopen-Datentyp
4	`MaxSubindex`	Maximalzahl Subindizes des Objekts
5	`ObjectType`	CANopen Object Type Code: 7 – Variable 8 – Array 9 – Record
6	`Name`	Name des Objekts

Aus Bild 4.66 lassen sich also `Index` = 0x1018, `DataType` = 0x0023 (`IDENTITY OBJECT`), `MaxSubindex` = 0x04, `ObjectType` = 0x09 (Record),

`Name = 49 64 65 6e 74 69 74 79` („*Identity*“) entnehmen. Die Länge des `Name`-Felds ergibt sich aus der Länge Mailbox Service Data (hier: `0x14` (20)).

Insbesondere ist nun bekannt, wie viele Subindizes das Objekt hat. Die Beschreibungen für die einzelnen Subindizes werden mit einem GetEntryDescription Request angefordert. Die SDO Information Service Data sind bei diesem Request wie folgt aufgebaut:

Tabelle 4.27: *SDO Information Service Data bei GetEntryDescription Request/Response*

Bytes	Parameter	Beschreibung
0-1	`Index`	Index des Objekts
2	`Subindex`	Subindex
3	`ValueInfo`	Enthält bitkodiert, welche Elemente zusätzlich angefordert werden bzw. in der Response vorhanden sind: **Bit** / **Bedeutung** 0 Access Rights 1 Object Category 2 Object in PDO mapbar? 3 Unit Type 4 Default Value 5 Minimum Value 6 Maximum Value
4-5	`DataType`	Datentyp (nur in Response)
6-7	`BitLength`	Bit-Länge des Objekts (nur in Response)
8-9	`ObjectAccess`	Zugriff in Abhängigkeit von ESM-Zustand (nur in Response) **Bit** / **Bedeutung** 0 Lesezugriff im Zustand `PRE-OPERATIONAL` 1 Lesezugriff im Zustand `SAFE-OPERATIONAL` 2 Lesezugriff im Zustand `OPERATIONAL` 3 Schreibzugriff im Zustand `PRE-OPERATIONAL` 4 Schreibzugriff im Zustand `SAFE-OPERATIONAL` 5 Schreibzugriff im Zustand `OPERATIONAL` 6 Objekt in RxPDO mapbar 7 Objekt in TxPDO mapbar 8 Objekt kann bei Backup (Store Parameters) gespeichert werden. 9 Objekt kann bei Setzen der Standard-Werte rückgesetzt werden.
10-...	`Data`	Nur in der Response vorhanden. Hier folgen die Werte zu den `ValueInfo` angekündigten Attributen. Ist die im Mailbox Header angegene Länge größer als die Länge, die durch die bisherigen Felder zu Stande kommt, folgt die Beschreibung als `char`-Array.

Der folgende Frame enthält den GetEntryDescription Request für Subindex 4 von Objekt `0x1018`. Dabei ist das Feld `ValueInfo` mit `00` belegt.


```
29 10 05 d1 e9 03 00 10 10 80 00 00 0a 00 00 00
00 03 00 80 05 00 00 00 18 10 04 00 00 00 05 00
e9 03 7f 10 01 00 00 00 00 00 00
```

Bild 4.67: *EtherCAT-Frame mit GetEntryDescription Request*

Die zugehörige GetEntryDescription Response ist im folgenden Frame enthalten:

```
8c 10 04 d2 e9 03 00 11 80 00 04 00 1d 00 00 00
00 03 00 80 06 00 00 00 18 10 04 00 07 00 20 00
07 00 53 65 72 69 61 6c 20 4e 75 6d 62 65 72 00
...
00 00 00 00 00 00 00 00 00 00 00 00 00 01 00
```

Bild 4.68: *EtherCAT-Frame mit GetEntryDescription Response*

Hier sind `ValueInfo = 00`, `DataType = 0x0007` (Unsigned32), `BitLength = 0x0020` (32), `ObjectAccess = 0x0007` (Lesezugriff in den Zuständen `PRE-OPERATIONAL`, `SAFE-OPERATIONAL`, `OPERATIONAL`). Die Mailbox Length ist mit `0x001d` (29) angegeben. Daher folgt noch die Beschreibung `53 65 72 69 61 6c 20 4e 75 6d 62 65 72` („*Serial Number*“).

4.20.4.2 SDO Services

SDO-Dienste werden zum Lesen (SDO Upload) und Schreiben (SDO Download) von Objekt-Werten benutzt. Die Protokollelemente sind direkt aus CANopen übernommen. Da CAN, der ursprüngliche „Träger“ für SDO-Dienste, eine Datenlänge von 8 Bytes unterstützt und eine SDO vier Bytes Header-Informationen benötigt, konnten in einer SDO maximal 4 Bytes Nutzdaten transportiert werden. Verwendet man bei EtherCAT den ***legacy Mode***, so gilt diese Einschränkung nach wie vor. Im ***enhanced Mode*** ist die Nutzdatenlänge dagegen nur noch durch die Größe der Mailbox begrenzt, was eine deutliche Beschleunigung des SDO-Verkehrs bedeutet.

Im legacy Mode wird bei einem SDO Upload/Download immer nur auf einen Subindex eines Objekts zugegriffen. Im enhanced Mode fällt auch diese Einschränkung weg. Im Regelfall können mit einem SDO alle Subindizes eines Objekts übertragen werden. Folgende SDO Services stehen zur Verfügung:

Tabelle 4.28: *SDO Services*

SDO Service	Beschreibung
SDO Download Expedited	Maximal 4 Bytes werden vom Client an den Server übertragen.
SDO Download Normal	Falls die Mailbox-Größe zur Übertragung der Daten ausreicht, werden diese in einem Request gesendet. Ansonsten wird ein segmentierter Transfer initiiert.
Download SDO Segment	Überträgt zusätzliche Daten, wenn die Mailbox-Größe nicht ausreicht, die Nutzdaten eines Downloads im Rahmen des SDO Download Normal zu übertragen.
SDO Upload Expedited	Maximal 4 Bytes werden vom Server an den Client übertragen.
SDO Upload Normal	Falls die Mailbox-Größe zur Übertragung der Daten ausreicht, werden

	diese in einem Request gesendet. Ansonsten wird ein segmentierter Transfer initiiert.
Upload SDO Segment	Überträgt zusätzliche Daten, wenn die Mailbox-Größe nicht ausreicht, die Nutzdaten eines Downloads im Rahmen des SDO Upload Normal zu übertragen.
Abort SDO Transfer	Im Fehlerfall sendet der Server anstelle der Response eine Abort-Message. Diese enthält Informationen zur Fehlerursache.

Die Kodierung der zugehörigen SDO Requests und Responses ist in Tabelle 4.29 dargestellt. In Klammern angegebene Werte sind bei der jeweiligen PDU fest. Die möglichen Werte für ein Feld sind immer nur beim ersten Vorkommen des Feldes erklärt.

Tabelle 4.29: *SDO-PDU*

Nachricht	**SDO Header**	
SDO Download Expedited Request	**Bytes**	**Beschreibung**
	0	**Bits** / **Bedeutung** 0: `SizeIndicator` (1) Wert 0: Datenlänge nicht angegeben Wert 1: Datenlänge folgt in `DataSetSize` 1: `TransferType` (1) Wert 0: Normal Transfer Wert 1: Expedited Transfer 2-3: `DataSetSize` Wert 0x00: 4 Bytes Wert 0x01: 3 Bytes Wert 0x02: 2 Bytes Wert 0x03: 1 Byte 4: `CompleteAccess` (0) Wert 0: Zugriff auf angesprochenen Index/Subindex Wert 1: Zugriff auf das ganze Objekt. `Subindex` muss 0 oder 1 sein, je nachdem, ob auch Subindex 0 übertragen werden soll oder nicht. 5-7: `CommandSpecifier` (0x01 - Download Request)
	1-2	`Index`
	3	`Subindex`
	4-7	**Daten**
SDO Download Expedited Response	**Bytes**	**Beschreibung**
	0	**Bits** / **Bedeutung** 0: `SizeIndicator` (0) 1: `TransferType` (0) 2-3: `DataSetSize` (0x00) 4: `CompleteAccess` (0) 5-7: `CommandSpecifier` (0x03 – Download Response)
	1-2	`Index`
	3	`Subindex`
	4-7	`reserved`
SDO Download Normal Request	**Bytes**	**Beschreibung**
	0	**Bits** / **Bedeutung** 0: `SizeIndicator` (1) 1: `TransferType` (0) 2-3: `DataSetSize` (0x00) 4: `CompleteAccess` 5-7: `CommandSpecifier` (0x01 – Download Request)
	1-2	`Index`
	3	`Subindex`
	4-7	`CompleteSize` – Gesamtlänge der Daten
	8-…	Daten. Falls (Länge der Mailbox Service Data – 10) ≥ `CompleteSize`, sind hier die gesamten Daten enthalten. Ansonsten folgt hier der erste Teil der Daten, und weitere Daten werden mit Download SDO Segment transportiert.
SDO Download Normal Response	**Bytes**	**Feld/Beschreibung**
	0	**Bits** / **Bedeutung** 0: `SizeIndicator` (0) 1: `TransferType` (0) 2-3: `DataSetSize` (0x00) 4: `CompleteAccess` (0) 5-7: `CommandSpecifier` (0x03 – Download Response)
	1-2	`Index`
	3	`Subindex`
	4-7	`reserved`
Download SDO Segment Request	**Bytes**	**Beschreibung**
	0	**Bits** / **Bedeutung** 0: `MoreFollows` Wert 0: Es folgen weitere Segmente. Wert 1: Letztes Segment 1-3: `SegDataSize` Wert 0x00: 7 Bytes Wert 0x01: 6 Bytes …: … Wert 0x07: 0 Bytes 4: `Toggle` Bit muss mit jedem Request getoggelt werden. Beim ersten Request: 0. 5-7: `CommandSpecifier` (0x00 – Download Segment Request)
	1…	Daten
Download SDO Segment Response	**Bytes**	**Beschreibung**
	0	**Bits** / **Bedeutung** 0: `MoreFollows` (0) 1-3: `SegDataSize` (0) 4: `Toggle`

			Wie im Request
		5-7	`CommandSpecifier` (0x03 – Download Segment Response)
SDO Upload Expedited Request	**Bytes**	**Beschreibung**	
	0	**Bits**	**Bedeutung**
		0	`SizeIndicator` (0)
		1	`TransferType` (1)
		2-3	`DataSetSize` (0x00)
		4	`CompleteAccess` (0)
		5-7	`CommandSpecifier` (0x02 – Upload Request)
	1-2	`Index`	
	3	`Subindex`	
SDO Upload Expedited Response	**Bytes**	**Beschreibung**	
	0	**Bits**	**Bedeutung**
		0	`SizeIndicator` (1)
		1	`TransferType` (1)
		2-3	`DataSetSize`
		4	`CompleteAccess` (0)
		5-7	`CommandSpecifier` (0x02 – Upload Response)
	1-2	`Index`	
	3	`Subindex`	
	4-7	Daten	
SDO Upload Normal Request	**Bytes**	**Beschreibung**	
	0	**Bits**	**Bedeutung**
		0	`SizeIndicator` (0)
		1	`TransferType` (0)
		2-3	`DataSetSize` (0x00)
		4	`CompleteAccess`
		5-7	`CommandSpecifier` (0x02 – Upload Request)
	1-2	`Index`	
	3	`Subindex`	
SDO Upload Normal Response	**Bytes**	**Beschreibung**	
	0	**Bits**	**Bedeutung**
		0	`SizeIndicator` (1)
		1	`TransferType` (0)
		2-3	`DataSetSize` (0x00)
		4	`CompleteAccess`
		5-7	`CommandSpecifier` (0x02 – Upload Response)
	1-2	`Index`	
	3	`Subindex`	
	4-7	`CompleteSize` – Gesamtlänge der Daten	
	8-…	Daten. Falls (Länge der Mailbox Service Data – 10) ≥ `CompleteSize`, sind hier die gesamten Daten enthalten. Ansonsten folgt hier der erste Teil der Daten, und weitere Daten werden mit Upload SDO Segment transportiert.	
Upload SDO Segment Request	**Bytes**	**Beschreibung**	
	0	**Bits**	**Bedeutung**
		0	`MoreFollows` (0)

		1-3	`SegDataSize` (0x00)
		4	`Toggle`
		5-7	`CommandSpecifier` (0x03 – Upload Segment Request)
Upload SDO Segment Response	**Bytes**	**Beschreibung**	
	0	**Bits**	**Bedeutung**
		0	`MoreFollows`
		1-3	`SegDataSize`
		4	`Toggle` Wie im Request
		5-7	`CommandSpecifier` (0x00 – Upload Segment Response)
	1-…	Daten	
Abort SDO Transfer Request	**Bytes**	**Beschreibung**	
	0	**Bits**	**Bedeutung**
		0	`SizeIndicator` (0)
		1	`TransferType` (0)
		2-3	`DataSetSize` (0x00)
		4	`Reserved` (0)
		5-7	`CommandSpecifier` (0x04 – Abort Transfer Request)
	1-2	`Index`	
	3	`Subindex`	
	4-7	`Abort Code`	

Tabelle 4.30 zeigt einen Teil der definierten Abort Codes.

Tabelle 4.30: *SDO Abort Codes (Auszug)*

Abort Code	Bedeutung
0x0503 0000	Toggle Bit nicht getoggelt
0x0504 0000	SDO Timeout
0x0601 0001	Leseversuch für Objekt, welches nur Schreibzugriff zulässt
0x0601 0002	Schreibversuch auf schreibgeschütztes Objekt
0x0602 0000	Objekt nicht im Objektverzeichnis

Acht Beispiele für SDO-PDUs

1. SDO Upload Expedited Request

```
29 10 05 cd e9 03 00 10 10 80 00 00 0a 00 00 00
00 03 00 20 40 00 10 00 00 00 00 00 00 00 05 00
e9 03 7f 10 01 00 00 00 00 00 00 00 00 00
```

Bild 4.69: *EtherCAT-Frame mit SDO Upload Expedited Request*

Der Mailbox Header (grau unterlegt) zeigt 10 (0x000a) Bytes Mailbox Service Data vom Type CoE (0x03) an. Laut CANopen Header (00 20) folgt ein SDO Request (siehe Tabelle 4.23). Dieser ist aufgrund der Längenangabe im Mailbox Header 8 Bytes lang (schwarz unterlegt). Der `CommandSpecifier` ist 0x02, also handelt es sich um einen SDO Upload Expedited Request. Diese betrifft den Index 0x1000 und den Subindex 0.

2. SDO Upload Expedited Response

Bild 4.70 zeigt die Slave-Antwort.

```
8c 10 04 ce e9 03 00 11 80 00 04 00 0a 00 00 00
00 03 00 30 43 00 10 00 00 00 00 00...
00 00 00 00 00 00 00 00 00 00 00 00 00 00 01 00
```

Bild 4.70: *EtherCAT-Frame mit SDO Upload Expedited Response*

Der CANopen Header (00 30) kündigt eine SDO Response (schwarz unterlegt) an. Die Binärdarstellung des ersten Bytes (0x43) ist 0100 0011, also handelt es sich um eine SDO Upload Expedited Response (CommandSpecifier = 0x02, TransferType = 1 (Expedited)) mit 4 Bytes Daten (DataSetSize = 00)). Es folgen Index = 0x1000, Subindex = 0x00 und Parameterwert (0x00000000).

3. SDO Upload Normal Request

Der folgende Frame enthält einen SDO Upload Normal Request für Index 0x100a, Subindex 0.

```
29 10 05 d7 e9 03 00 10 10 80 00 00 0a 00 00 00
00 03 00 20 40 0a 10 00 00 00 00 00 ...
```

Bild 4.71: *EtherCAT-Frame mit SDO Upload Normal Request*

4. SDO Upload Normal Response

Die zugehörige Response ist in folgendem Frame enthalten:

```
8c 10 04 d8 e9 03 00 11 80 00 04 00 1c 00 00 00
00 03 00 30 41 0a 10 00 12 00 00 00 6e 65 74 58
20 45 76 61 6c 20 46 69 72 6d 77 61 72 65 ...
```

Bild 4.72: *EtherCAT-Frame mit SDO Upload Normal Response*

Hier kündigt der Mailbox Header 28 (0x001c) Bytes an Mailbox Service Data an. Die SDO Response ist also 26 Bytes lang. Deren erstes Byte (0x41) ist binär 0100 0001. Der CommandSpecifier ist wieder 0x02, also liegt eine SDO Upload Response vor. Im Unterschied zur Response in Beispiel 2 ist hier jedoch der TransferType = 0 (Normal Transfer), d. h. es handelt sich um eine SDO Upload Normal Response. Es folgt also nach Index = 0x100a und Subindex = 0x00 die CompleteSize mit 0x00000012. Die 18 Bytes Daten sind 6e 65 74 58 20 45 76 61 6c 20 46 69 72 6d 77 61 72 65 („*netX Eval Firmware*“).

5. SDO Download Expedited Request

```
37 10 04 cc e9 03 00 11 02 80 00 00 00 00 00 00
05 00 e9 03 00 10 10 80 00 00 0a 00 00 00 00 03
00 20 2f 02 20 00 58 00 00 00 00 00 05 00 e9 03
7f 10 01 00 00 00 00 00 00
```

Bild 4.73: *EtherCAT-Frame mit SDO Download Expedited Request*

Das erste Byte im SDO Request (0x2f) lautet binär 0010 1111. Der CommandSpecifier ist somit (0x01), also liegt ein SDO Download Request vor. Der

TransferType ist 1, also handelt es sich um einen SDO Download Expedited Request. Wegen SegDataSize = 0x03 ist die Datenlänge 1 Byte. Der Wert 0x58 soll in Subindex 0x00 von Objekt 0x2002 geschrieben werden.

6. Abort SDO Transfer Request

Die Slave-Antwort ist in Bild 4.74 zu sehen:

```
8c 10 04 cd e9 03 00 11 80 00 04 00 0a 00 00 00 00
03 00 30 80 02 20 00 02 00 01 06...
```

Bild 4.74: *EtherCAT-Frame mit Abort SDO Transfer Request*

Das erste Byte der SDO Response (0x80) lautet binär 1000 0000. Der CommandSpecifier 0x04 zeigt einen Abort SDO Transfer Request an. Nach Index = 0x2002 und Subindex = 0x00 folgt der Abort Code 0x0601 0002. Laut Tabelle 4.30 handelt es sich um einen Schreibversuch auf das schreibgeschützte Objekt 0x2002, Subindex 0.

7. SDO Download Expedited Request

```
37 10 04 8f e9 03 00 11 02 80 00 00 00 00 00 00
05 00 e9 03 00 10 10 80 00 00 0a 00 00 00 00 03
00 20 2f 02 20 03 01 00 00 00 00 00 05 00 e9 03
7f 10 01 00 00 00 00 00 00
```

Bild 4.75: *EtherCAT-Frame mit SDO Download Expedited Request*

Wie oben handelt es sich um einen SDO Download Expedited Request. Ein Byte (0x01) soll in Subindex 0x03 von Objekt 0x2002 geschrieben werden.

8. Init SDO Download Response

Die Slave-Antwort ist in folgendem Frame enthalten:

```
8c 10 04 8e e9 03 00 11 80 00 04 00 06 00 00 00
00 03 00 30 62 02 20 03 ...
```

Bild 4.76: *EtherCAT-Frame mit SDO Download Expedited Response*

Der CommandSpecifier lautet 0x03, also handelt es sich um eine Download Response. Der Schreibzugriff war erfolgreich.

4.20.4.3 SDO-Downloads beim Startup

Im **TwinCAT System Manager** können in der Registerkarte **Startup** SDO Downloads eingebaut werden, die in bestimmten Zustandsübergängen zusätzlich ausgeführt werden. Auf diese Weise können Initialisierungen im Objektverzeichnis durchgeführt werden. Betätigt man die Schaltfläche **Neu...**, so erscheint ein Dialog, bei dem das zu schreibende Objekt ausgewählt werden kann:

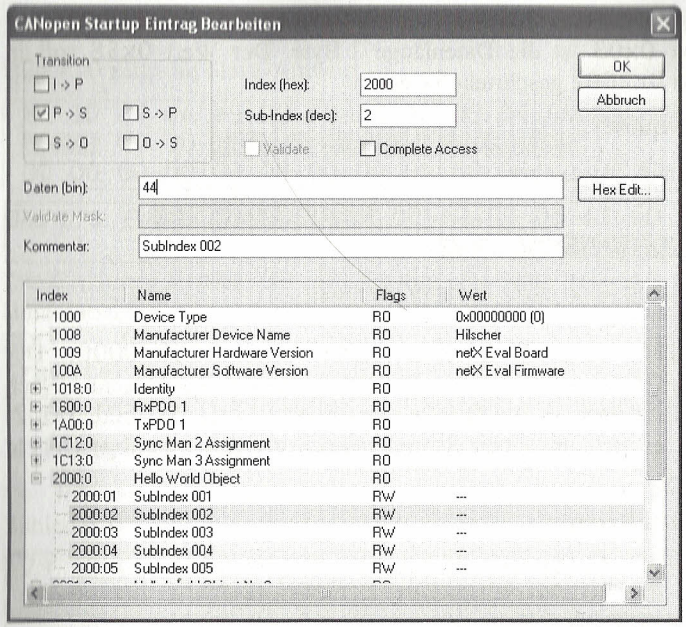


Bild 4.77: *SDO Download in der Startup-Sequenz*

Zusätzlich können unter **Transition** die Zustandsübergänge, bei denen der SDO Download erfolgen soll, spezifiziert werden.

4.20.5 Ethernet over EtherCAT (EoE)

Um Ethernet-Frames mit TCP/IP-Protokollelemente über EtherCAT zu übertragen, wird ebenfalls die Mailbox genutzt. In diesem Fall enthält das Feld `Type` im Mailbox Header den Wert `0x02` (vgl. Tabelle 4.22). Ethernet-Frames werden getunnelt, d. h. der komplette Ethernet-Frame (ohne Päambel und CRC) wird direkt in den Datenteil der Mailbox eingebettet.

Zwei Anwendungsfälle sind bei EoE zu unterscheiden. Verfügt der EtherCAT-Slave über einen TCP/IP-Protokollstack, um z. B. einen Web-Server zu realisieren, so ist der Slave selbst Quelle bzw. Ziel der TCP/IP-PDUs. EtherCAT kann aber auch als Transitstrecke für Ethernet-Frames genutzt werden. In diesem Fall wird über einen Switch-Port eines EtherCAT-Slaves ein Anschluss für Nicht-EtherCAT-Geräte bereitgestellt, über den Ethernet-Frames in das EtherCAT-System „eingeschleust" werden können. Die Ethernet-Frames werden als Nutzlast der Mailbox transportiert.

4.20.5.1 Fragmentierung

Aufgrund des Overheads, den die diversen Header im EtherCAT-Frame verursachen, kann Ethernet-Frame maximaler Länge nicht im Datenteil der Mailbox untergebracht werden. Es ist auch nicht immer sinnvoll, die Mailbox so groß anzulegen, dass sie die Kapazität

des Ethernet-Frames voll ausschöpft. Die Übertragung eines Ethernet-Frames maximaler Länge nimmt ca. 125 µs in Anspruch. Damit wären Prozessdaten-Zykluszeiten von 100 µs nicht mehr realisierbar. Aus diesem Grunde werden die Mailbox nicht allzu groß angelegt und eine Fragmentierung der Ethernet-Frames durchgeführt. Hierfür bietet EoE einen **EoE Fragment Request**. Beim Empfänger werden die Fragmente wieder zusammengefügt.

4.20.5.2 Adressierung

Wie in 4.20.2.1 bemerkt, spielen MAC-Adressen bei der EtherCAT-Kommunikation keine Rolle. Damit ein Ethernet-Knoten auf einen Web-Server zugreifen kann, der in einem EtherCAT-Slave integriert ist, muss der betreffende EtherCAT-Slave jedoch über MAC-Adresse und IP-Adresse ansprechbar sein. Mit dem EoE-Dienst **Set IP Parameter** können dem EtherCAT-Slave Parameter wie (virtuelle) MAC-Adresse und IP-Adresse zugewiesen werden. Diese Parameter sind auch im CANopen-Objektverzeichnis eines EtherCAT-Slaves abgelegt. Hierfür stehen die EtherCAT-spezifischen Objekte `0x1110` („*Virtual MAC Address*") und `0x1111` („*IP Adress Info*") zur Verfügung.

Der EtherCAT-Master fungiert als virtueller Ethernet-Switch, indem er Ethernet-Frames, die sich an die entsprechende Adresse richten, in das EtherCAT-System einschleust.